



**Atatürk
University**

**Atatürk University-Faculty of Engineering
Department of Computer Engineering
Fall Semester-2025**

**MBI203-Data Structures and Algorithms
Asst. Prof. Esra ÇELİK**

**Conditional Staircase Data Structure – CSDS
230709028-Zeynep YILMAZ**

The Purpose of Conditional Staircase Data Structure (CSDS)

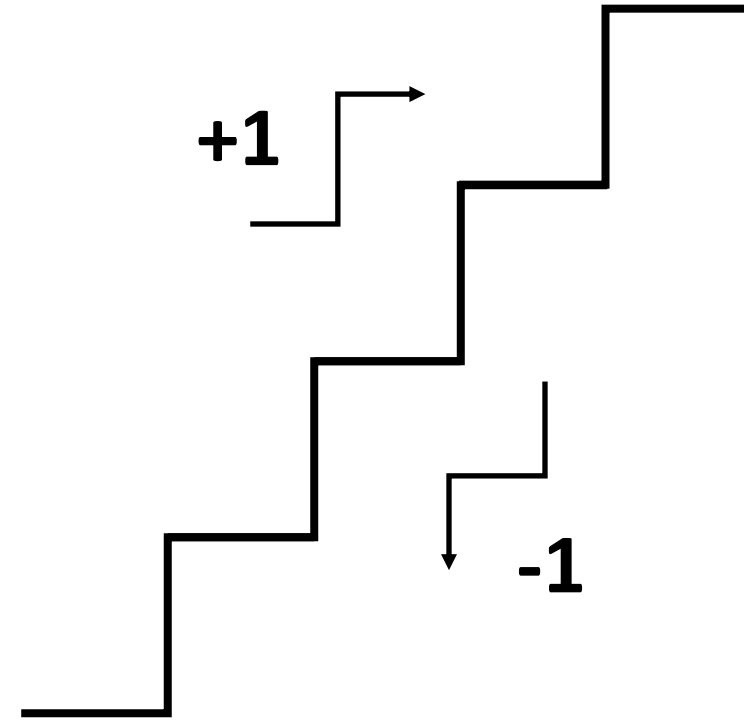
- This data structure is a **special structure** that allows progression through steps but allows the user to **skip more steps when certain conditions are met**.
- Its primary purpose is as follows:
 - “The user can always proceed to the next step or step below with normal steps; however, when a predetermined condition is met, it provides an **escalator-like function**, allowing the user to ascend or descend **multiple steps at once**.”
 - Furthermore, the decision whether this jump can be performed or not is controlled by the **control mechanism (Rule Engine)** which **evaluates the predetermined state of each step before activating the jump**.
 - This structure is particularly used in systems that require **gradual progression and conditional rapid jumps**.

Working Principle

- CSDS consists of a series of steps.
- Each node contains:
 - Step number: **stepNo**
 - Pointer to the next step: **up**
 - Pointer to the next step: **down**
 - Jump link: **jumpTarget**
 - Jump length: **jumpLength**
 - Required condition for using jump: **requiredCase**
 - Before a jump is confirmed, **a rule checking mechanism (Rule Engine)** evaluates this **requiredCase** **to decide whether the jump can be activated.**

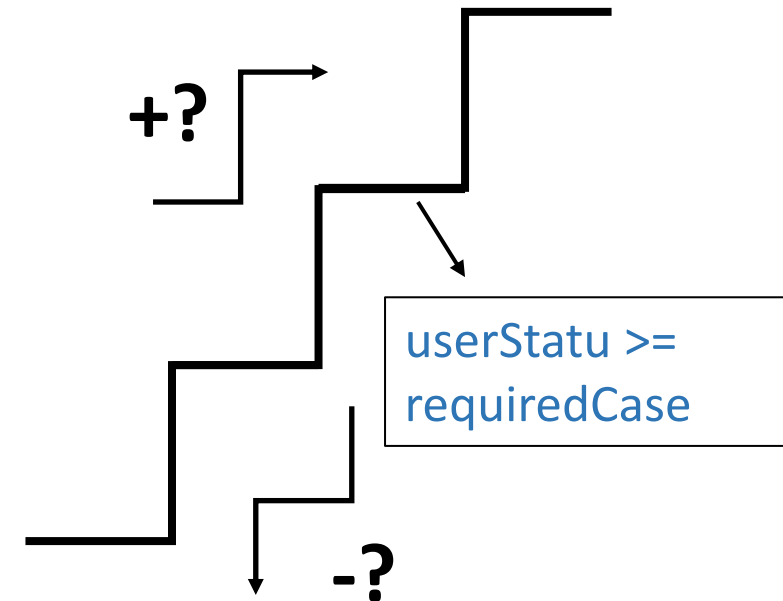
Working Principle-Normal Progress

- The user can always move like a **normal staircase**.
- The up pointer can be used to **move to a higher step(+1)**, and the down pointer can be used to **move to a lower step(-1)**.
- **No conditions** are required for these movements.
- There are no conditions required for these moves, and the **Rules Engine doesn't work** here because it only checks conditions for jump attempts, not the normal step-by-step movement.



Working Principle-Conditional Jump (Escalator Use)

- Some steps have a jumpTarget defined; this represents a special function that **skips multiple steps from that step**.
- When the user wants to use this jump, the condition is first checked:
- If **userStatu >= requiredCase**, the jump is activated, and the user moves to the jumpTarget step in one step.
- If the condition is not met, the jump is not performed at all; the user simply continues with the normal +1 / -1 steps.
- Before the jump is confirmed, this situation is evaluated by a rule checking mechanism (**Rule Engine**) and it is decided **whether the jump will be confirmed or not**.



Example uses from computer science

- **In Learning Management Systems (LMS):**
 - You can advance to higher-level classes with a placement test.
- **Game Level Progression:**
 - With the required score, you can advance to the next part of the game.
- **Task Flow Systems:**
 - Authorized users can skip certain tasks.
- **In Artificial Intelligence Training Pipelines:**
 - Quality-driven advancements can be achieved in AI training processes.
- **Level-Based Data Processing:**
 - Quality-dependent layer jumps can be enabled in data processing pipelines.

Examples of usage from daily life

- **Escalator – Normal Stair System:**
 - Progress is easier and faster when there is an elevator.
- **Skipping a Grade in Education:**
 - One can advance to the next grade with sufficient success.
- **Career Ladders:**
 - Promotion can be achieved through a special examination or experience.
- **Gradual Course Programs:**
 - You can advance to higher classes in the courses by taking a placement test.
- **Subscription / Membership Systems:**
 - It can reach higher levels with subscriptions.

Basic Case:

- **firstStep:** The bottom rung of the ladder
- **lastStep:** The top rung of the ladder
- **current:** The step the user is currently on
- **jumpTarget:** The target of the jump link
- **requiredCase:** The condition at that step

Basic Operations

- `createStaircase()` : Create the staircase
- `insertStep()` : Adding steps in between
- `deleteStep()` : Delete step
- `moveUp()` : +1 progress
- `moveDown()` : -1 regression
- `canJump()` : Rule Engine: condition check
- `tryJump()` : Attempt to jump (if there is a jump, the condition is met)
- `traverseFromBottom()` / `traverseFromTop()`: Printing steps on the screen

Create the Conditional Staircase Data Structure

Node Structure

```
struct Step {  
    int stepNo;  
    Step* up;  
    Step* down;  
  
    Step* jumpTarget;  
    int jumpLength;  
    int requiredCase;
```

```
Step(int no)  
{  
    stepNo = no;  
    up = nullptr;  
    down = nullptr;  
    jumpTarget = nullptr;  
    jumpLength = 0;  
    requiredCase = 0;  
}  
};
```

Create the Conditional Staircase Data Structure

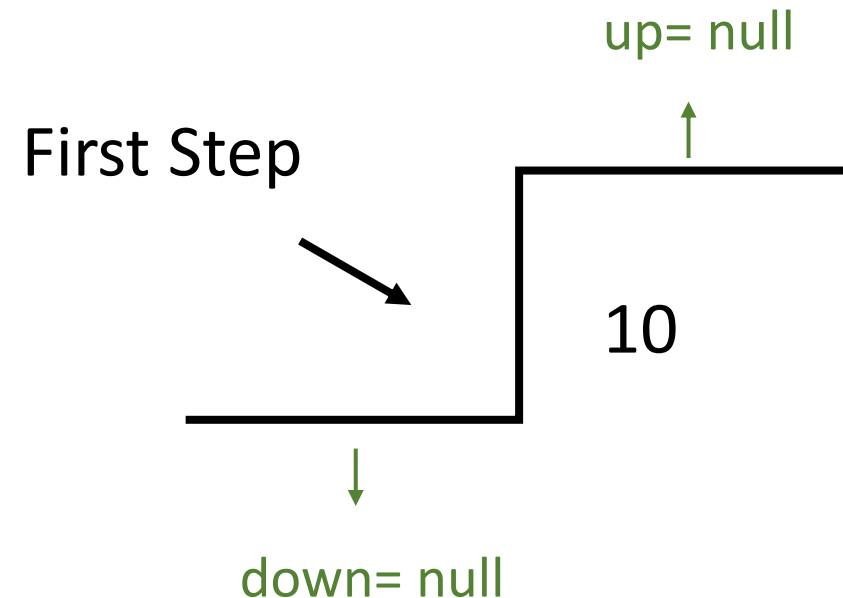
createStaircase

```
void createStaircase(int n) {  
    firstStep = nullptr;  
    lastStep = nullptr;  
    current = nullptr;  
    Step* prev = nullptr;  
  
    for (int i = 1; i <= n; i++) {  
        int no;  
        cout << i << ". enter a value  
for                the step: ";  
        cin >> no;  
  
        Step* node = new Step(no);  
  
        if (firstStep == nullptr) {  
            firstStep = node; }  
  
        if (prev != nullptr) {  
            prev->up = node;  
            node->down = prev;  
        }  
  
        prev = node;  
    }  
  
    lastStep = prev;  
    current = firstStep;  
}
```

createStaircase

```
Step* node = new Step(10);  
if (firstStep == nullptr) {  
    firstStep = node; }  
  
if (prev != nullptr) {  
    prev->up = node;  
    node->down = prev;  
}  
  
prev = node;  
}
```

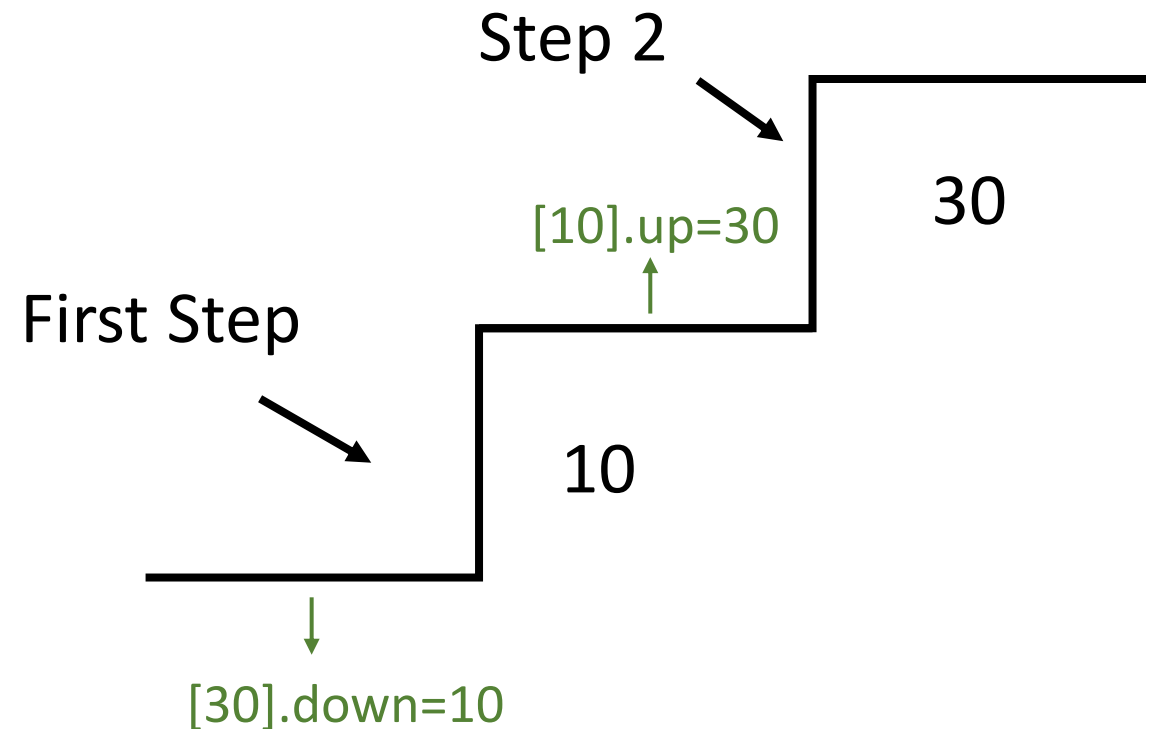
firstStep = 10
lastStep = null
current = null
prev = 10



createStaircase

```
Step* node = new Step(30);  
if (firstStep == nullptr) {  
    firstStep = node; }  
  
if (prev != nullptr) {  
    prev->up = node; // [10].up=30  
    node->down = prev; //  
[30].down=10  
}  
  
prev = node; // prev = 30  
}
```

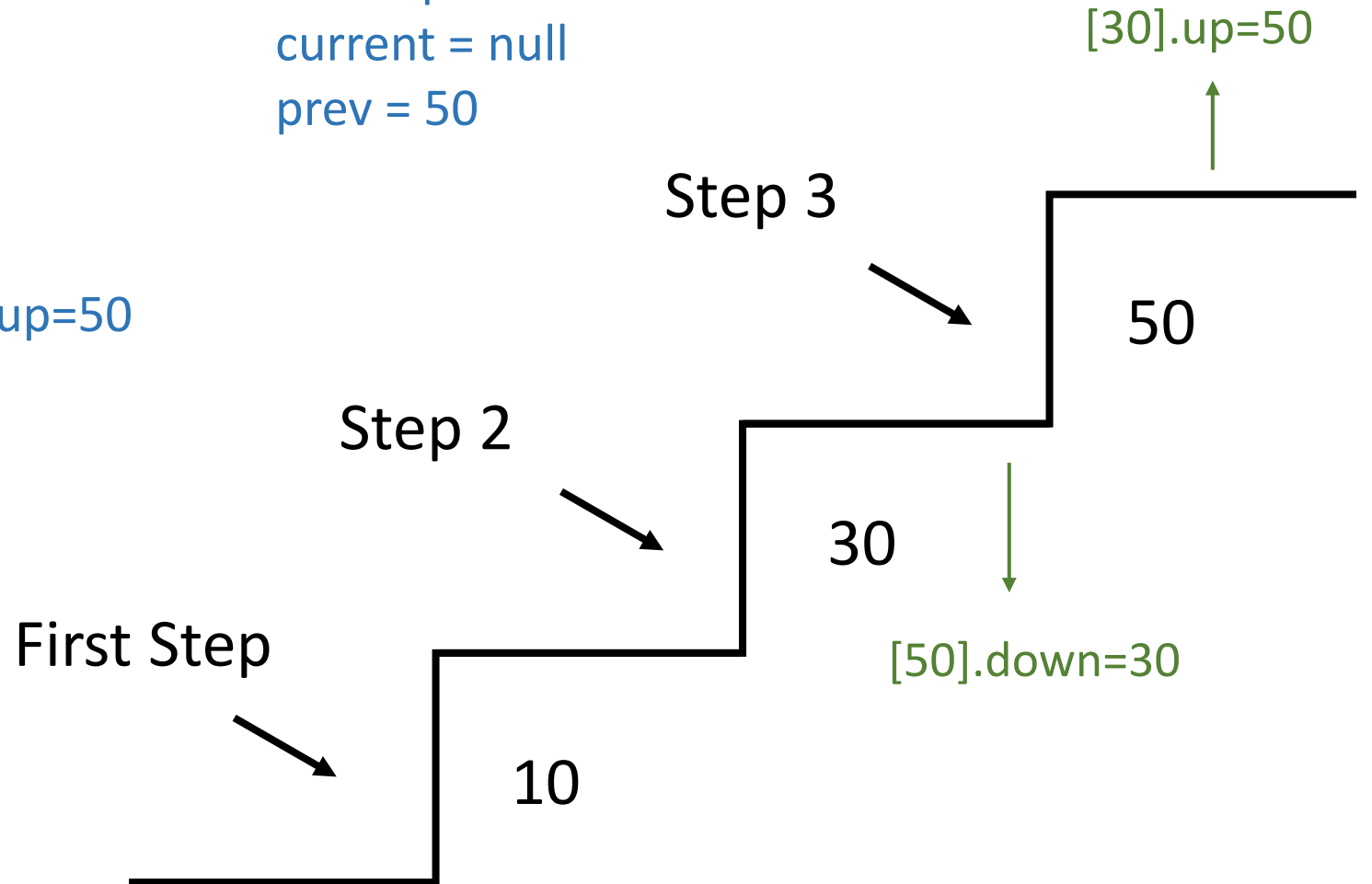
```
firstStep = 10  
lastStep = null  
current = null  
prev = 30
```



createStaircase

```
Step* node = new Step(50);  
if (firstStep == nullptr) {  
    firstStep = node; }  
  
if (prev != nullptr) {  
    prev->up = node; // [30].up=50  
    node->down = prev; //  
[50].down=30  
}  
  
prev = node; // prev = 50  
}
```

firstStep = 10
lastStep = null
current = null
prev = 50



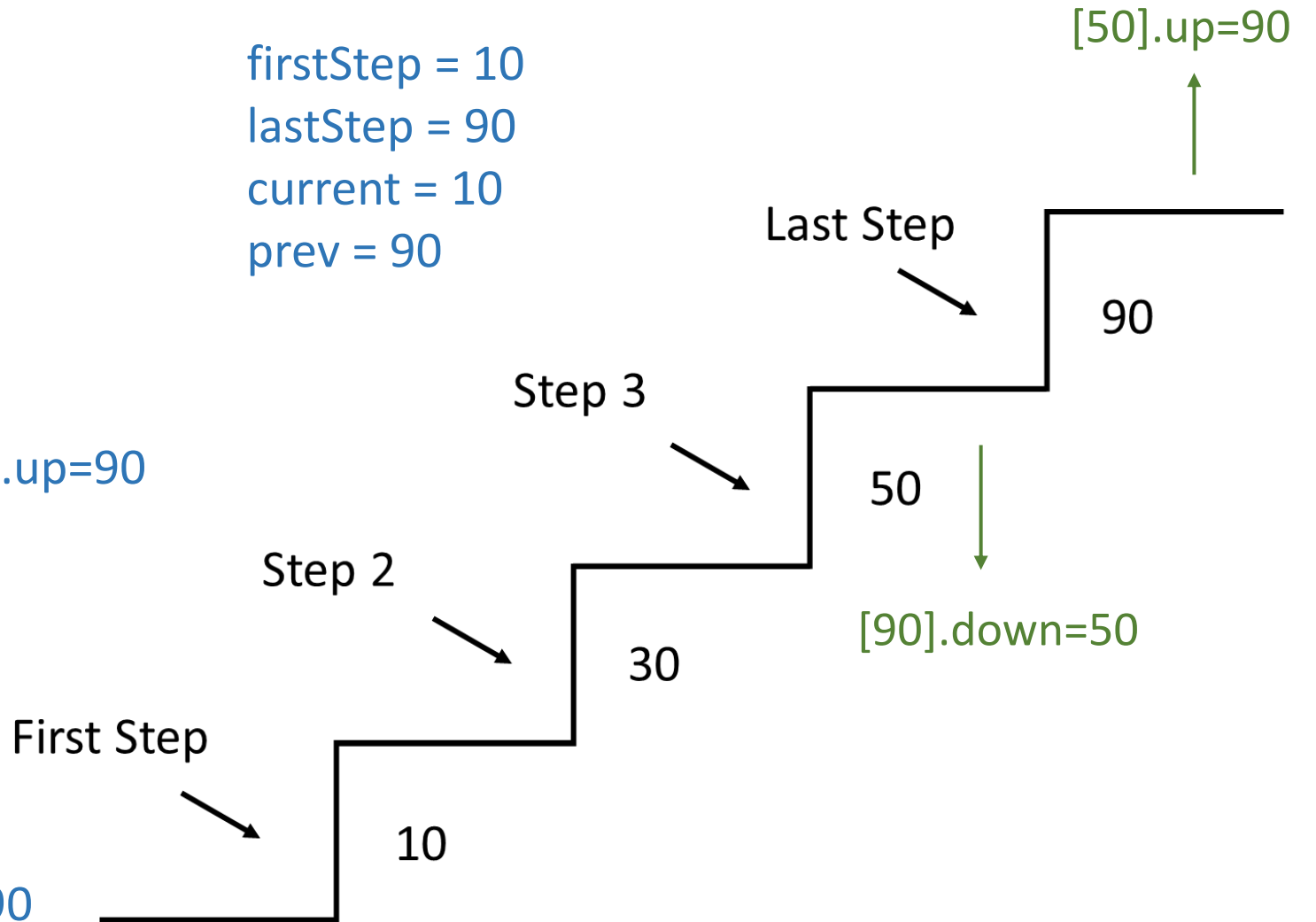
createStaircase

```
Step* node = new Step(90);
if (firstStep == nullptr) {
    firstStep = node; }

if (prev != nullptr) {
    prev->up = node; // [50].up=90
    node->down = prev; //
[90].down=50
}

prev = node; // prev = 90
}
lastStep = prev; // lastStep = 90
current = firstStep; // current = 10
}
```

firstStep = 10
lastStep = 90
current = 10
prev = 90



Insert Data

```
void insertStep(int afterStepNo, int
newStepNo)  {
    Step* goalStep = findStep(afterStepNo);

    if (goalStep == nullptr) {
        cout << "Step " << afterStepNo << " not
found. Cannot insert.\n";
        return;
    }

    Step* newNode = new Step(newStepNo);
    Step* nextNode = goalStep->up;
```

```
goalStep->up = newNode;
    newNode->down = goalStep;
    newNode->up = nextNode;

    if (nextNode != nullptr)
    {
        nextNode->down = newNode;
    }
    else
    {
        lastStep = newNode;
    }
    cout << "Inserted step " << newStepNo
<< " after step " << afterStepNo << ".\n"; }
```

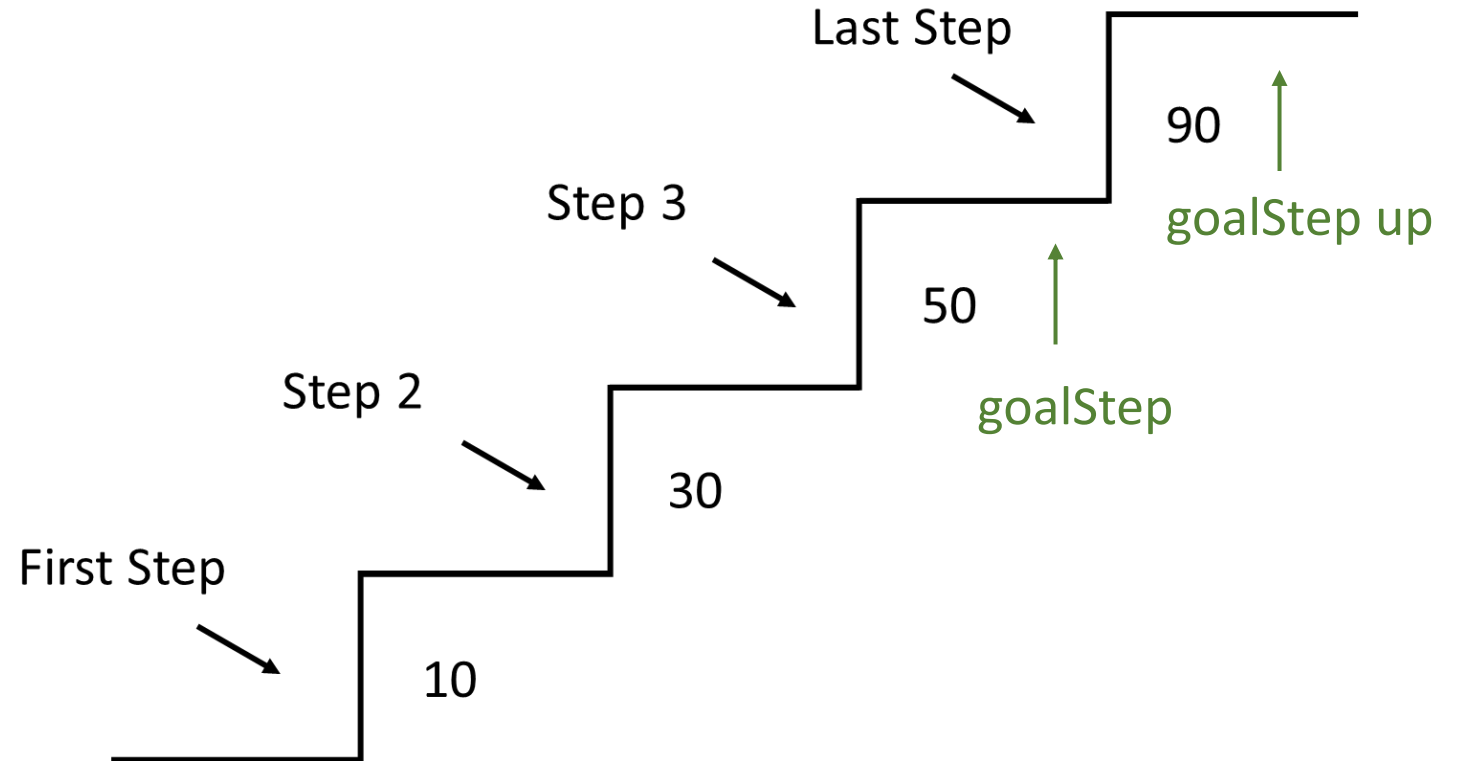
insertStep(50,70)

```
Step* goalStep = findStep(50);
```

```
Step* newNode = new  
Step(70);
```

```
Step* nextNode = goalStep-  
>up;
```

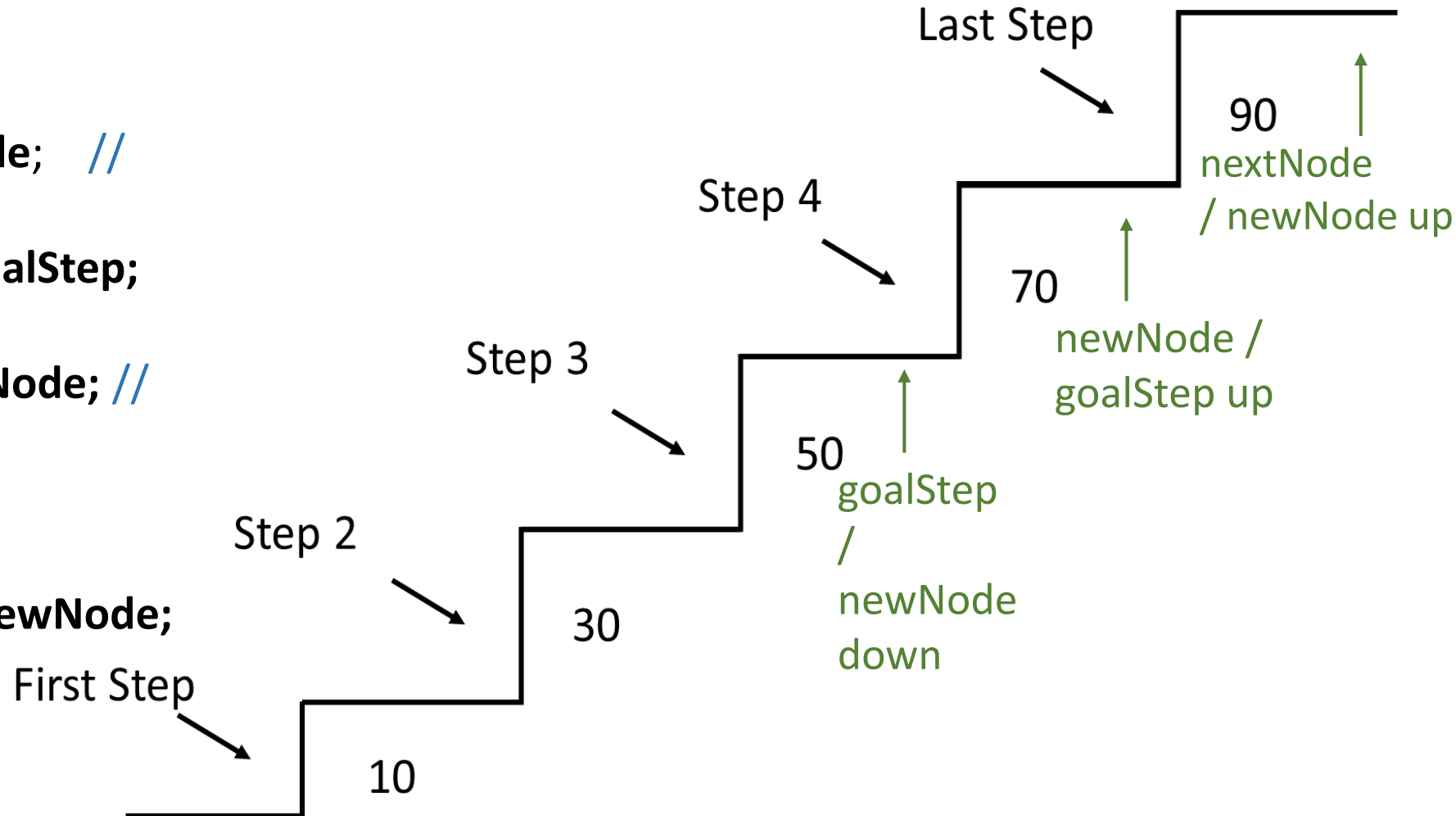
```
// nextNode = 70
```



insertStep(50,70)

```
goalStep->up = newNode; //  
[50].up = 70  
newNode->down = goalStep;  
// [70].down = 50  
newNode->up = nextNode; //  
[70].up = 90
```

```
nextNode->down = newNode;  
// [90].down = 70
```



Delete Data

```
void deleteStep(int stepNo)    {
    Step* node = findStep(stepNo);
    if (node == nullptr)
    {
        cout << "Step " << stepNo << " not
found. Cannot delete.\n";
        return;
    }
    Step* prev = node->down;
    Step* next = node->up;
```

```
    if (prev != nullptr)
        prev->up = next;
    else
        firstStep = next;

    if (next != nullptr)
        next->down = prev;
    else
        lastStep = prev;

    if (current == node)    {
        if (next != nullptr)
            current = next;
        else
            current = prev;    }
    delete node;
```

deleteStep(30)

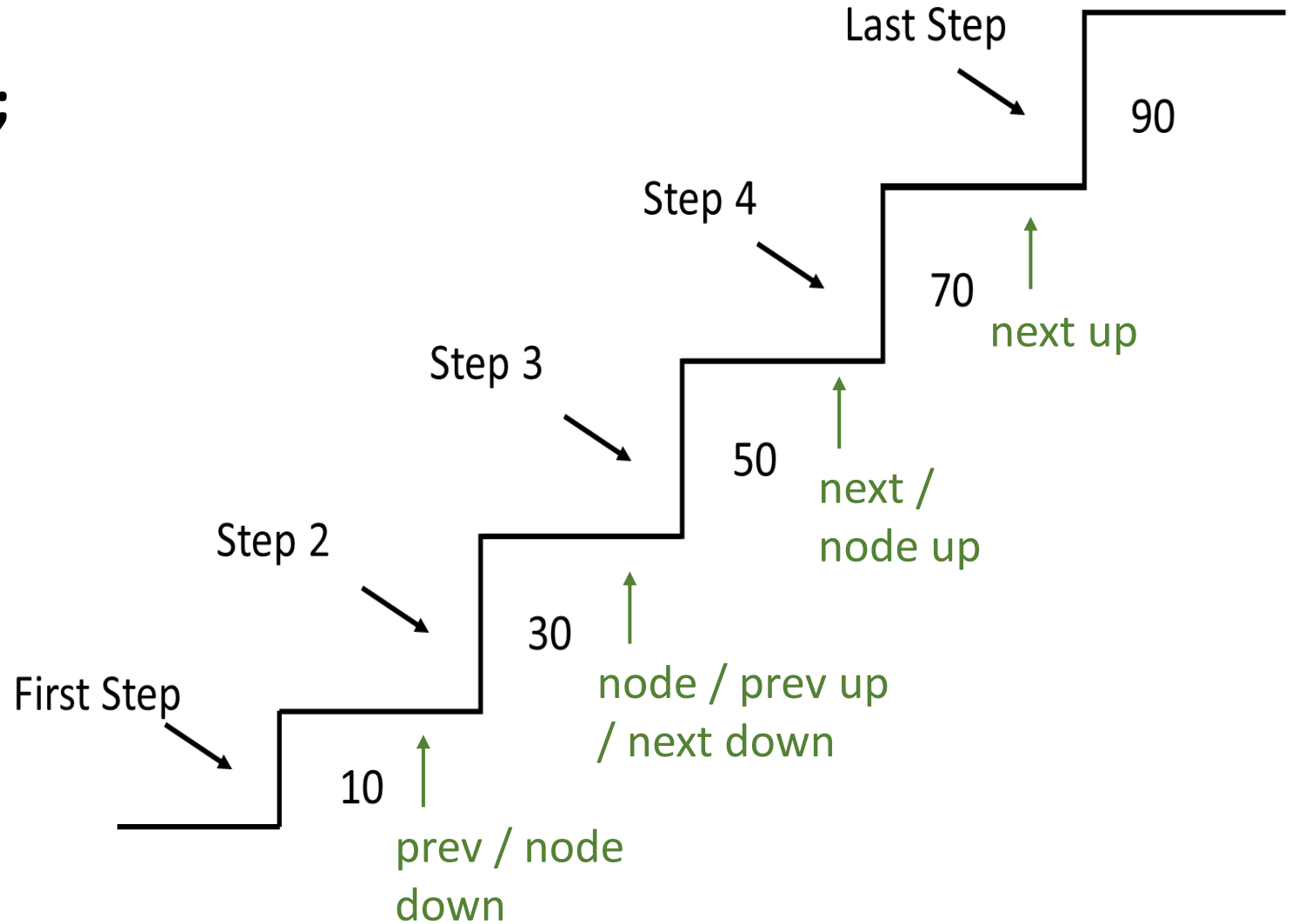
```
Step* node = findStep(30);
```

```
Step* prev = node->down;
```

```
// prev = 10
```

```
Step* next = node->up;
```

```
// next = 50
```



deleteStep(30)

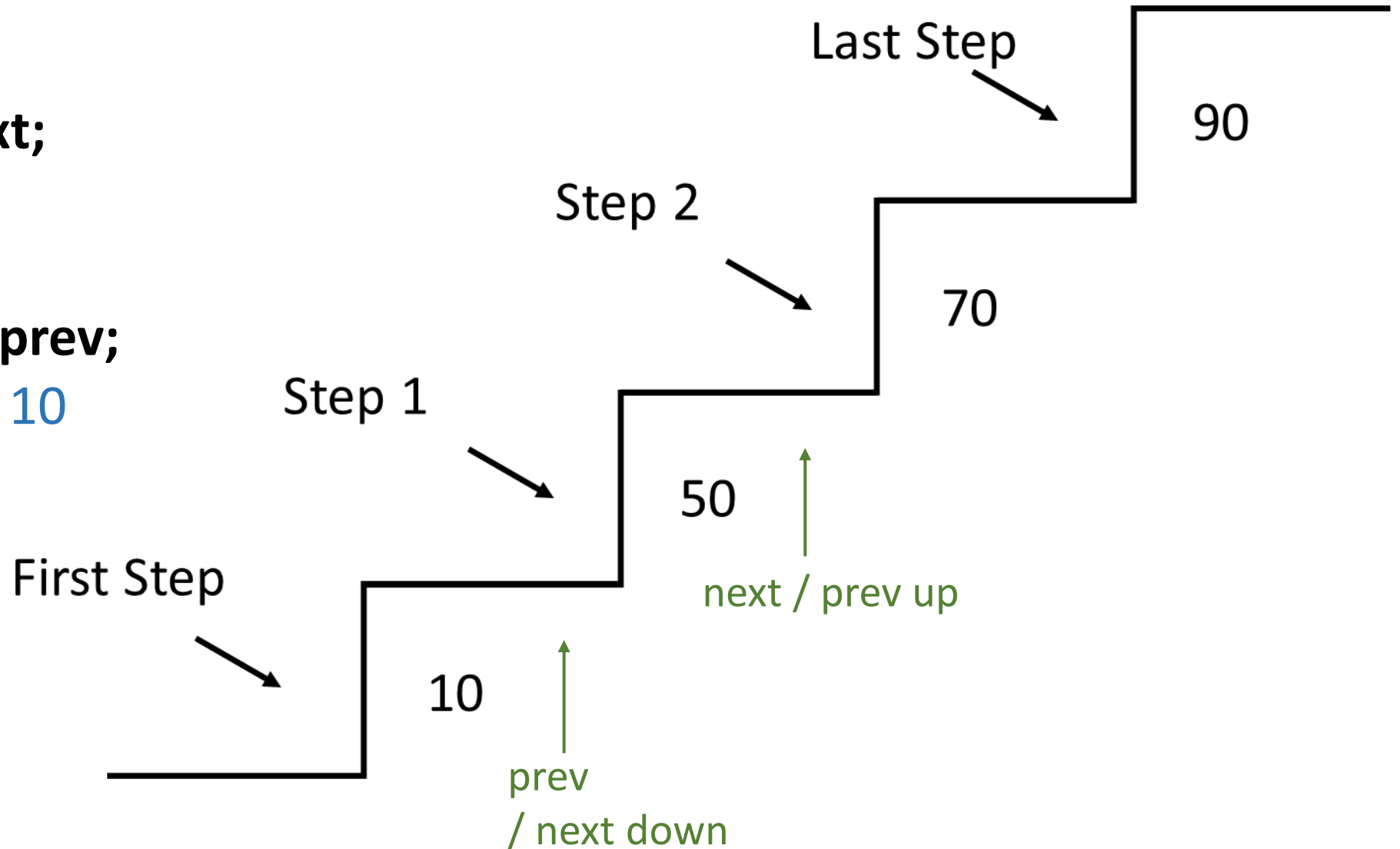
prev->up = next;

// [10].up = 50

next->down = prev;

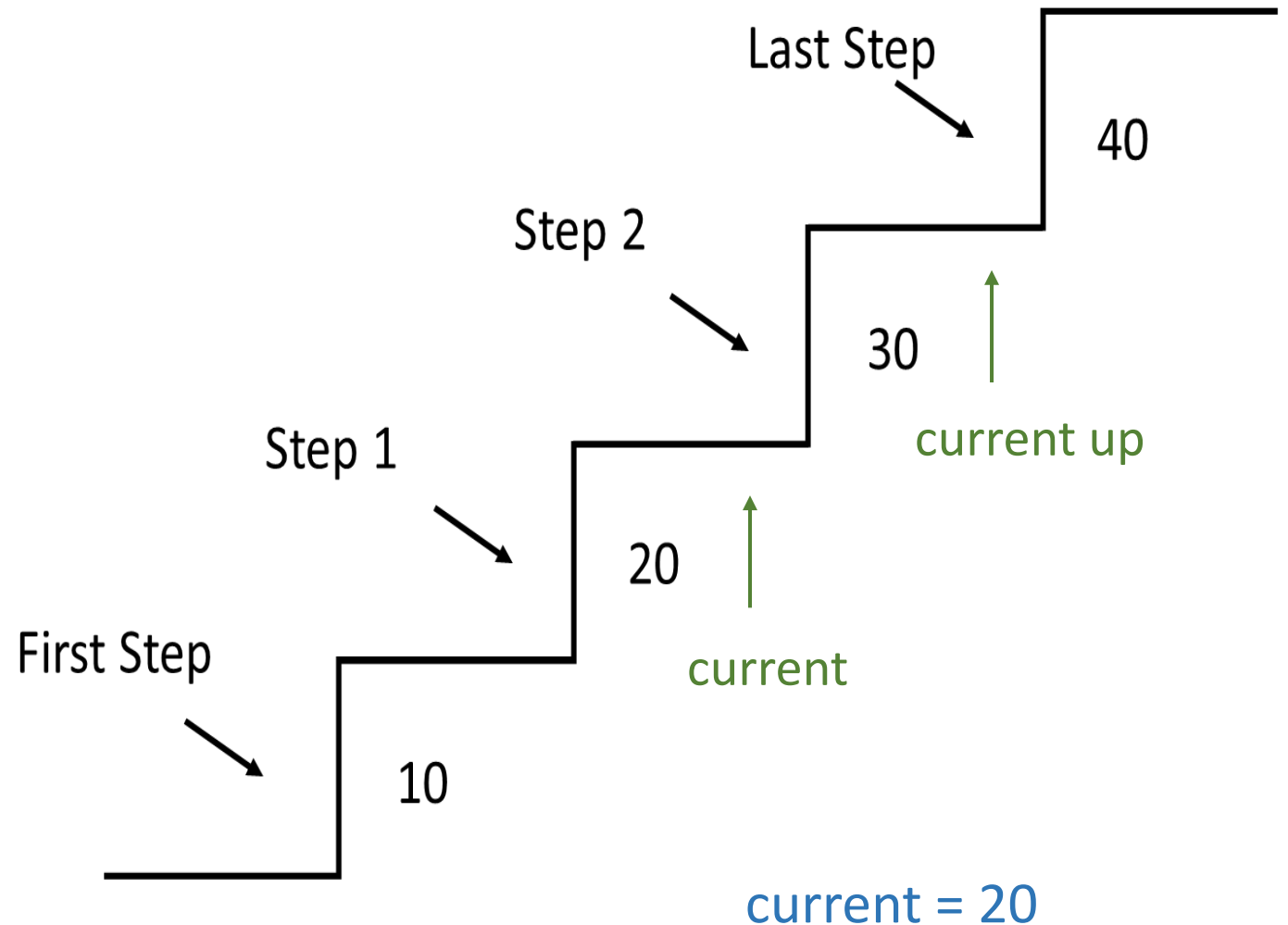
// [50].down = 10

delete node;



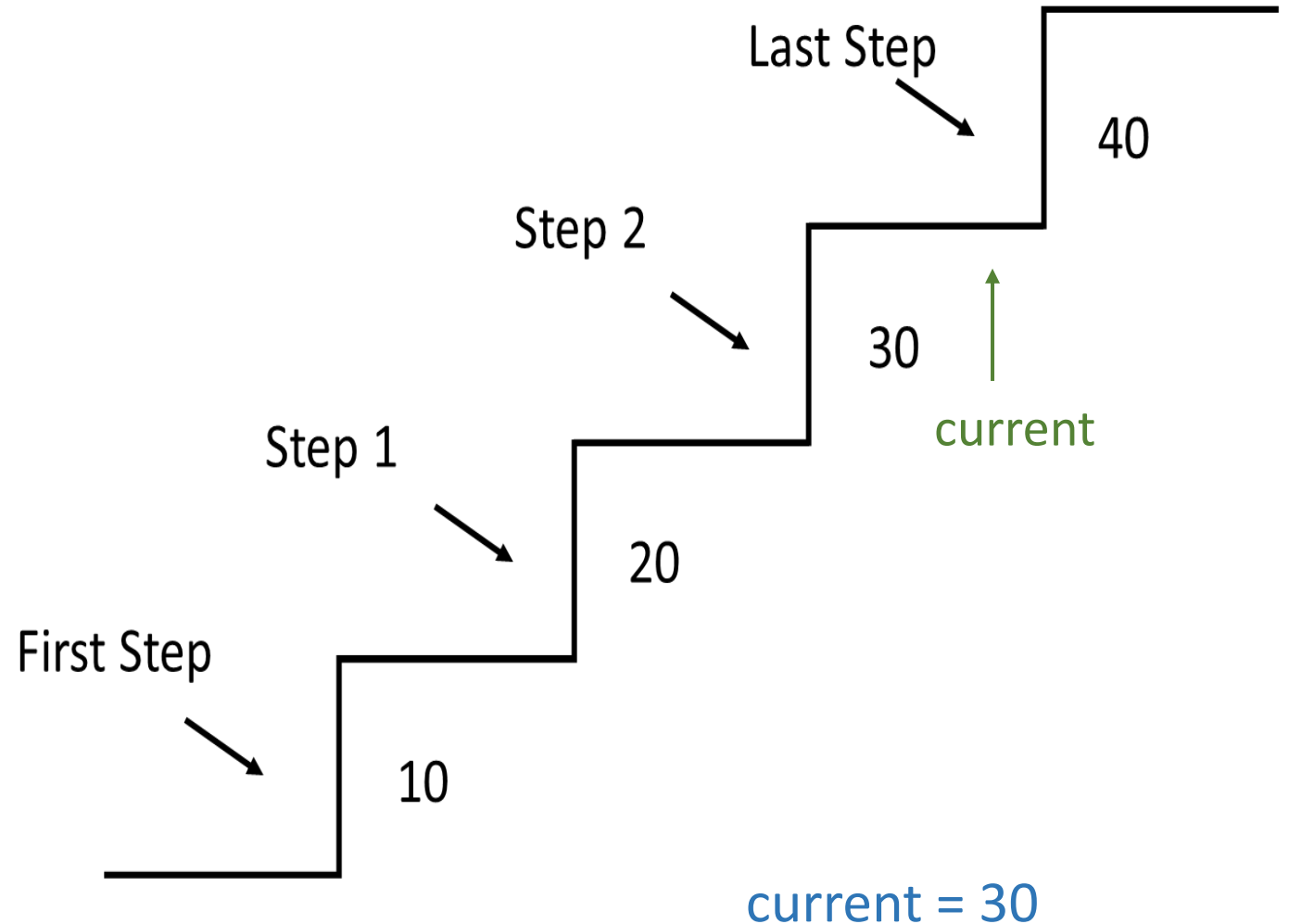
Move Operations (moveUp)

```
void moveUp() {  
    if (current != nullptr &&  
        current->up != nullptr)  
    {  
        current = current->up;  
        cout << "Moved up.  
Current step: " << current->  
stepNo << "\n";  
    }  
    else  
    {  
        cout << "Cannot move  
up. Already at the top or  
current is null.\n"; } } }
```



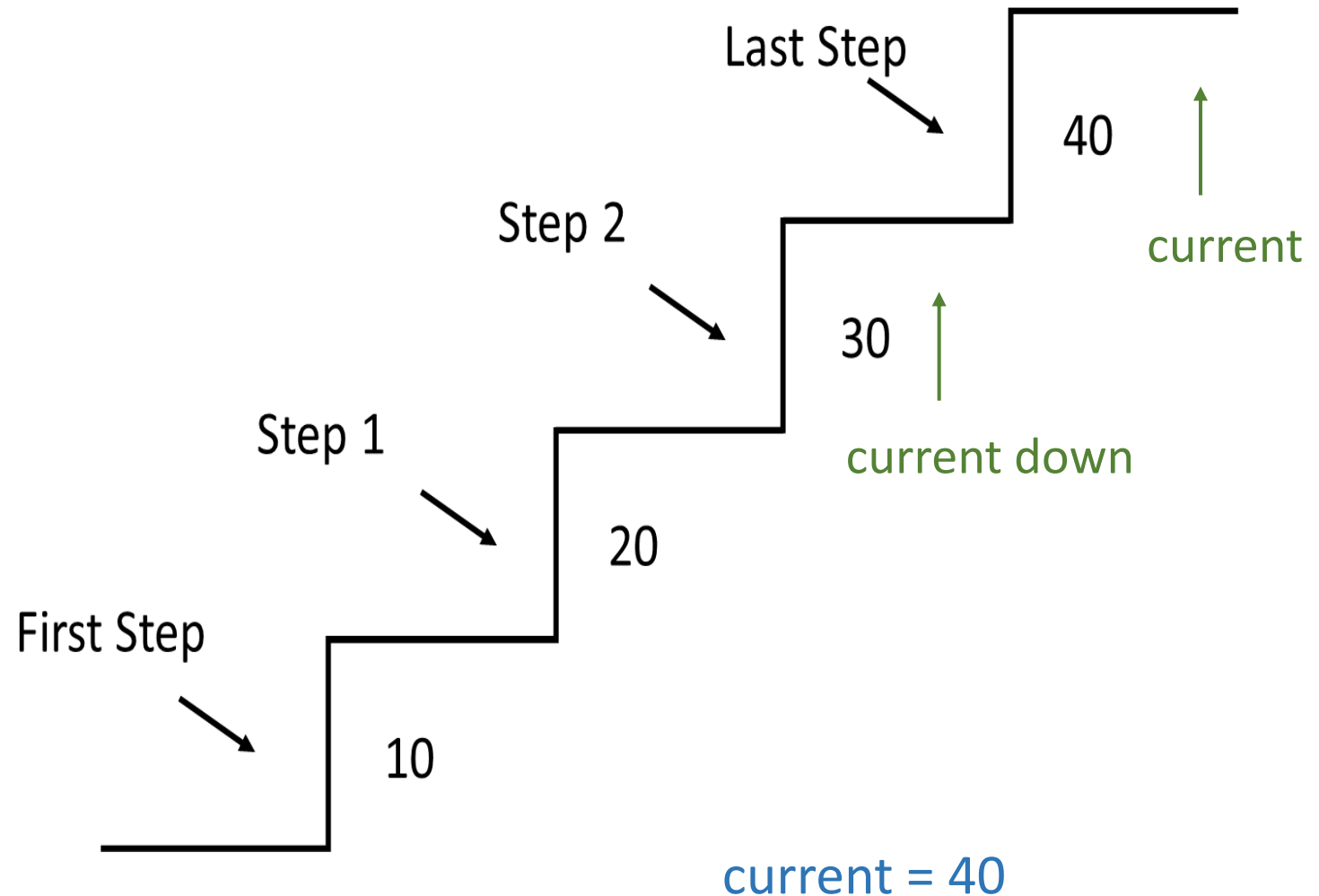
moveUp

```
void moveUp() {  
    if (current != nullptr &&  
        current->up != nullptr)  
    {  
        current = current->up;  
        cout << "Moved up.  
Current step: " << current->  
stepNo << "\n";  
    }  
    else  
    {  
        cout << "Cannot move  
up. Already at the top or  
current is null.\n"; } } }
```



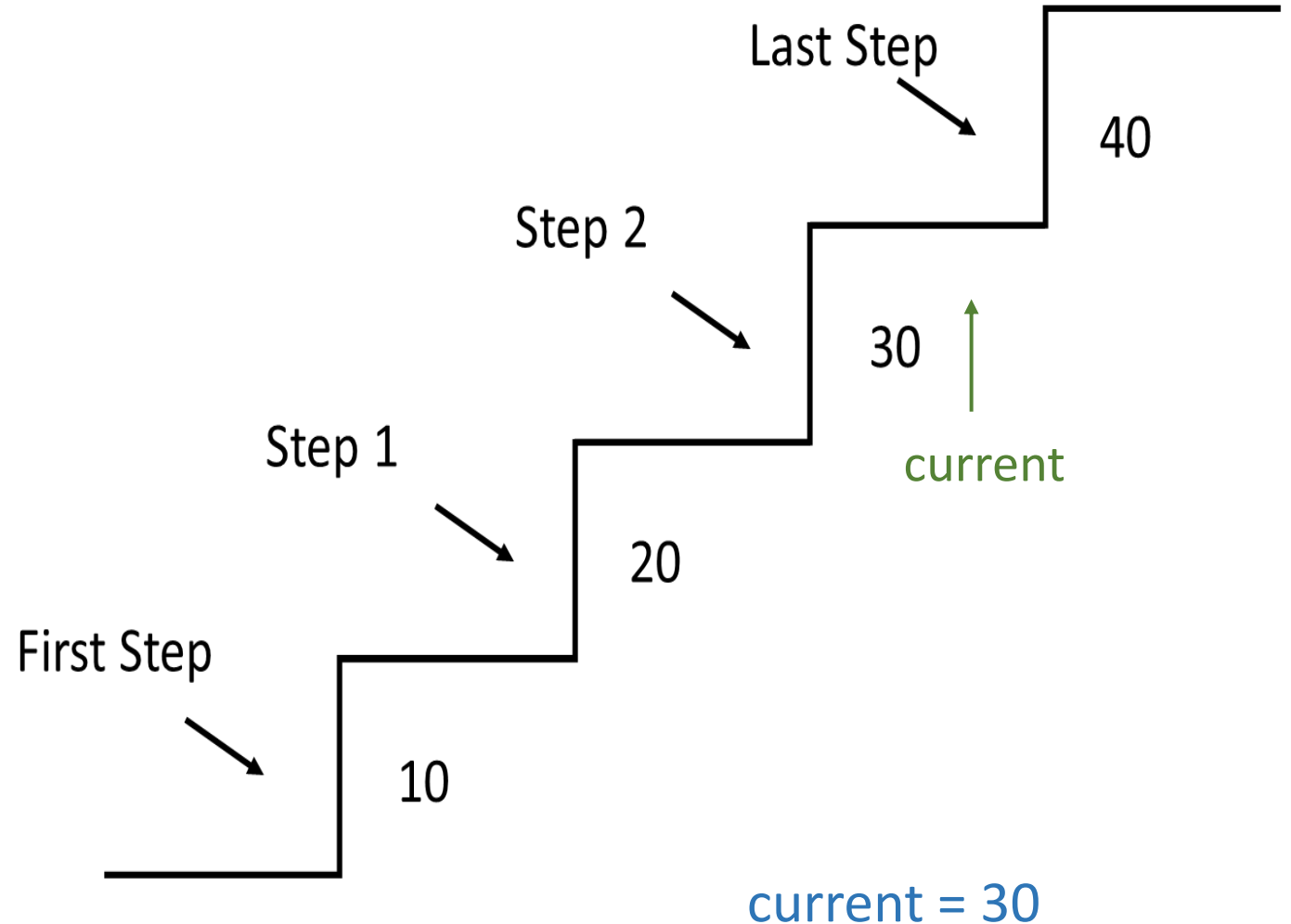
Move Operations (moveDown)

```
void moveDown() {  
    if (current != nullptr &&  
        current->down != nullptr)  
    {  
        current = current->down;  
        cout << "Moved down.  
Current step: " << current->  
stepNo << "\n";  
    }  
    else  
    {  
        cout << "Cannot move  
down. Already at the bottom  
or current is null.\n";  
    }  
}
```



moveDown

```
void moveDown() {  
    if (current != nullptr &&  
        current->down != nullptr)  
    {  
        current = current->down;  
        cout << "Moved down.  
Current step: " << current-  
>stepNo << "\n";  
    }  
    else  
    {  
        cout << "Cannot move  
down. Already at the bottom  
or current is null.\n"; } } }
```



Conditional Jump (setJump)

```
void setJump(int startStepNo, int
targetStepNo, int requiredCase)
{
    Step* startStep =
findStep(startStepNo);
    Step* targetStep =
findStep(targetStepNo);

    if (startStep == nullptr || targetStep ==
nullptr)
    {
        cout << "Step not found. Cannot set
jump.\n";
        return;
    }
```

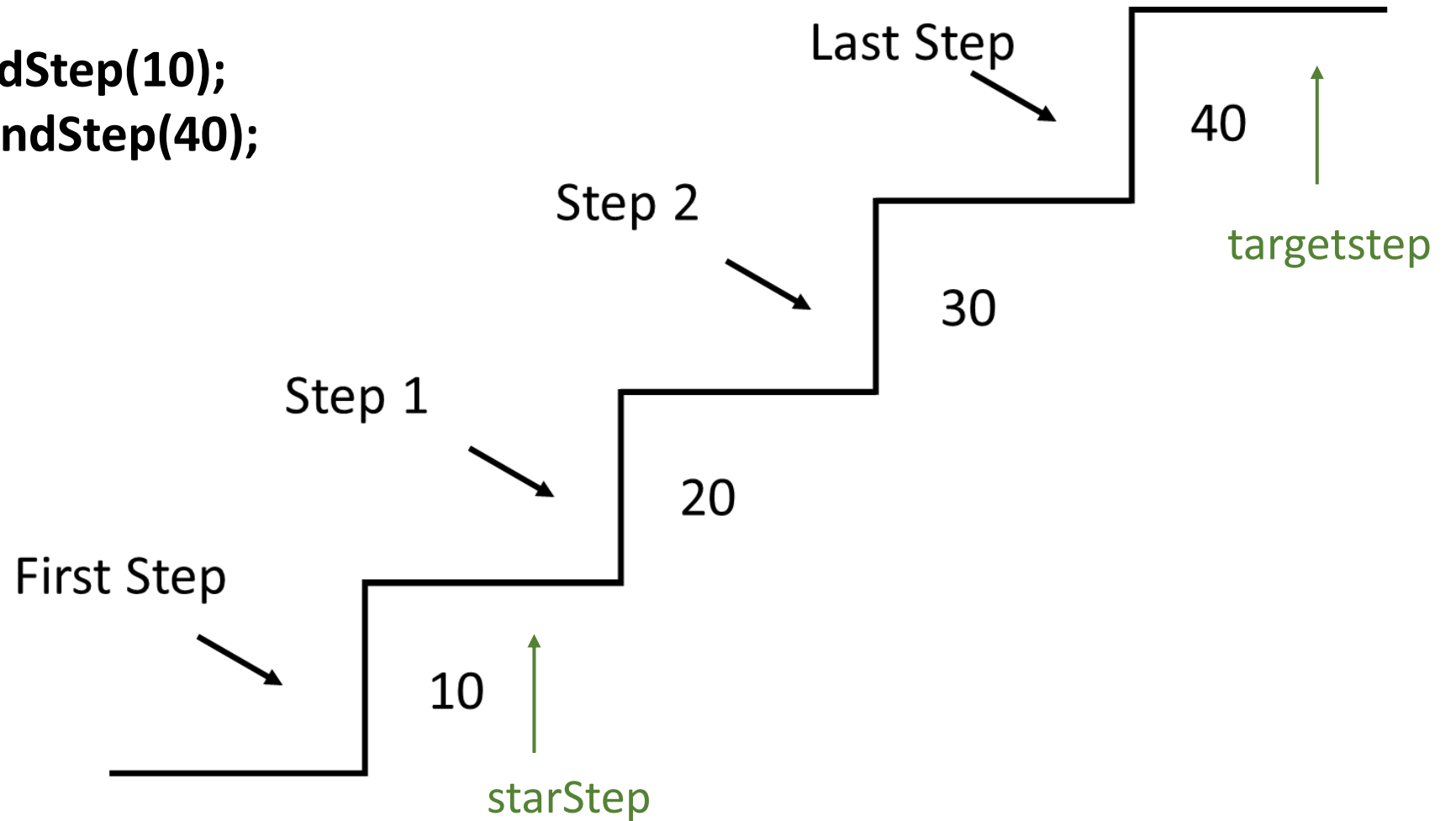
```
    if (startStep == targetStep) {
        cout << "Start and target steps cannot be
the same.\n";
        return;
    }

    startStep->jumpTarget = targetStep;
    startStep->jumpLength = targetStep->stepNo
- startStep->stepNo;
    startStep->requiredCase = requiredCase;

    cout << "Jump created from step " <<
startStepNo
        << " to step " << targetStepNo
        << " (condition = " << requiredCase <<
").\n";
}
```

setJump(10,40,5)

```
Step* startStep = findStep(10);  
Step* targetStep = findStep(40);
```

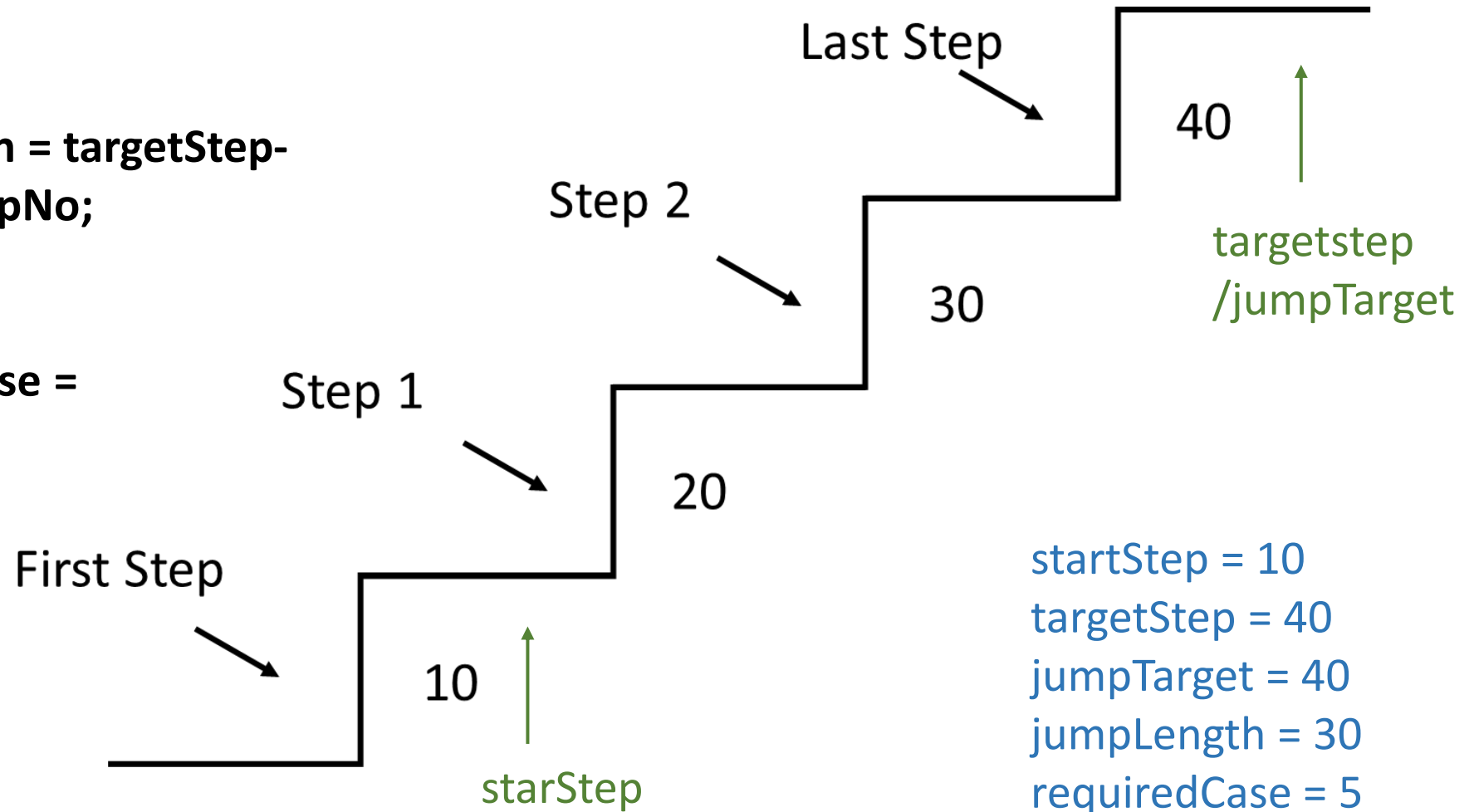


setJump(10,40,5)

```
startStep->jumpTarget = targetStep;  
// 10 -> 40
```

```
startStep->jumpLength = targetStep-  
>stepNo - startStep->stepNo;  
// 30 = 40 - 10
```

```
startStep->requiredCase =  
requiredCase;  
// userStatu >= 5
```



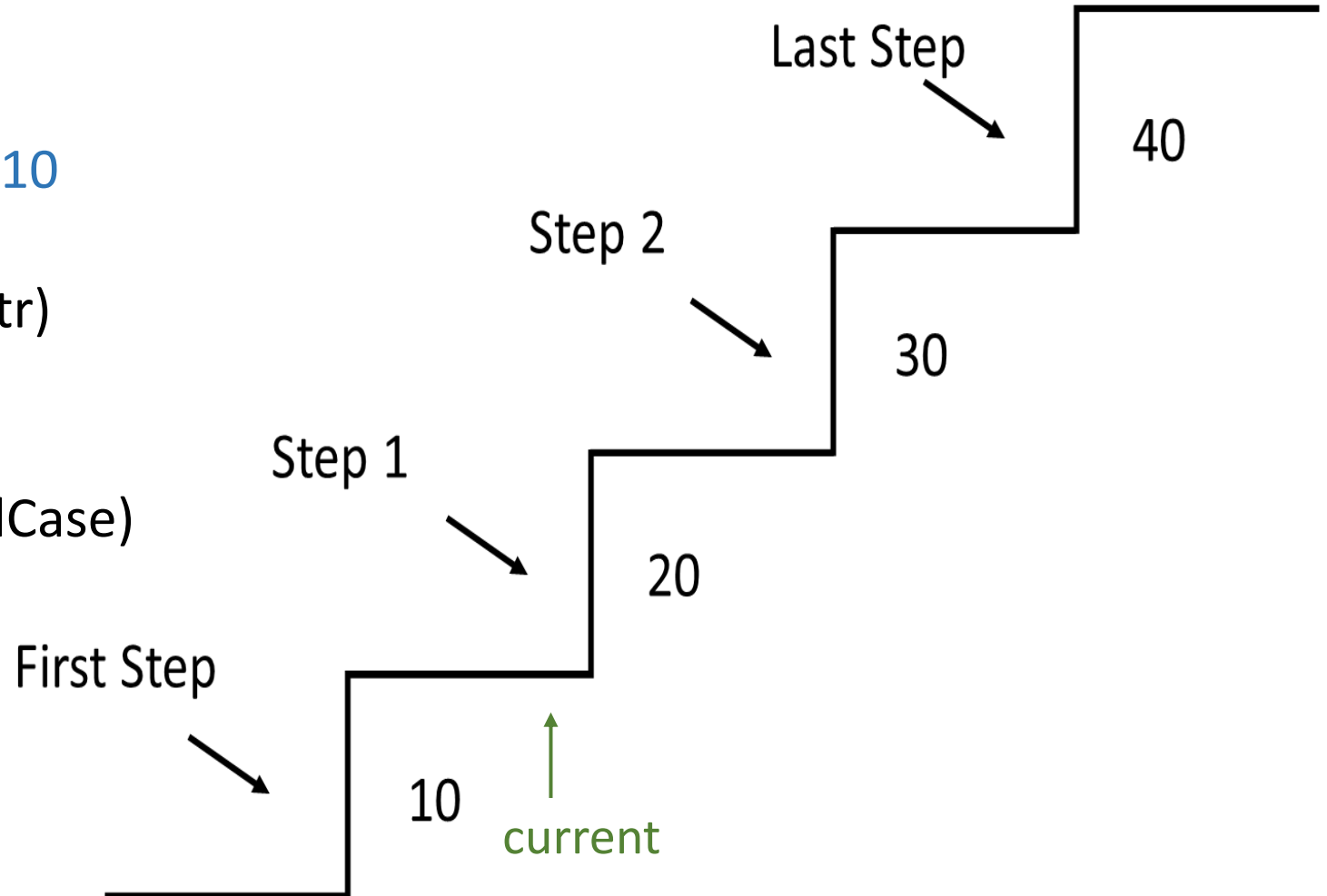
Conditional Jump (canJump)

```
bool canJump(Step* step, int userStatu)
{
    if (step == nullptr)
        return false; // current = 10

    if (step->jumpTarget == nullptr)
        return false;

    if (userStatu < step->requiredCase)
        return false;

    return true; }
```



canJump

```
bool canJump(Step* step, int userStatu)
```

```
{
```

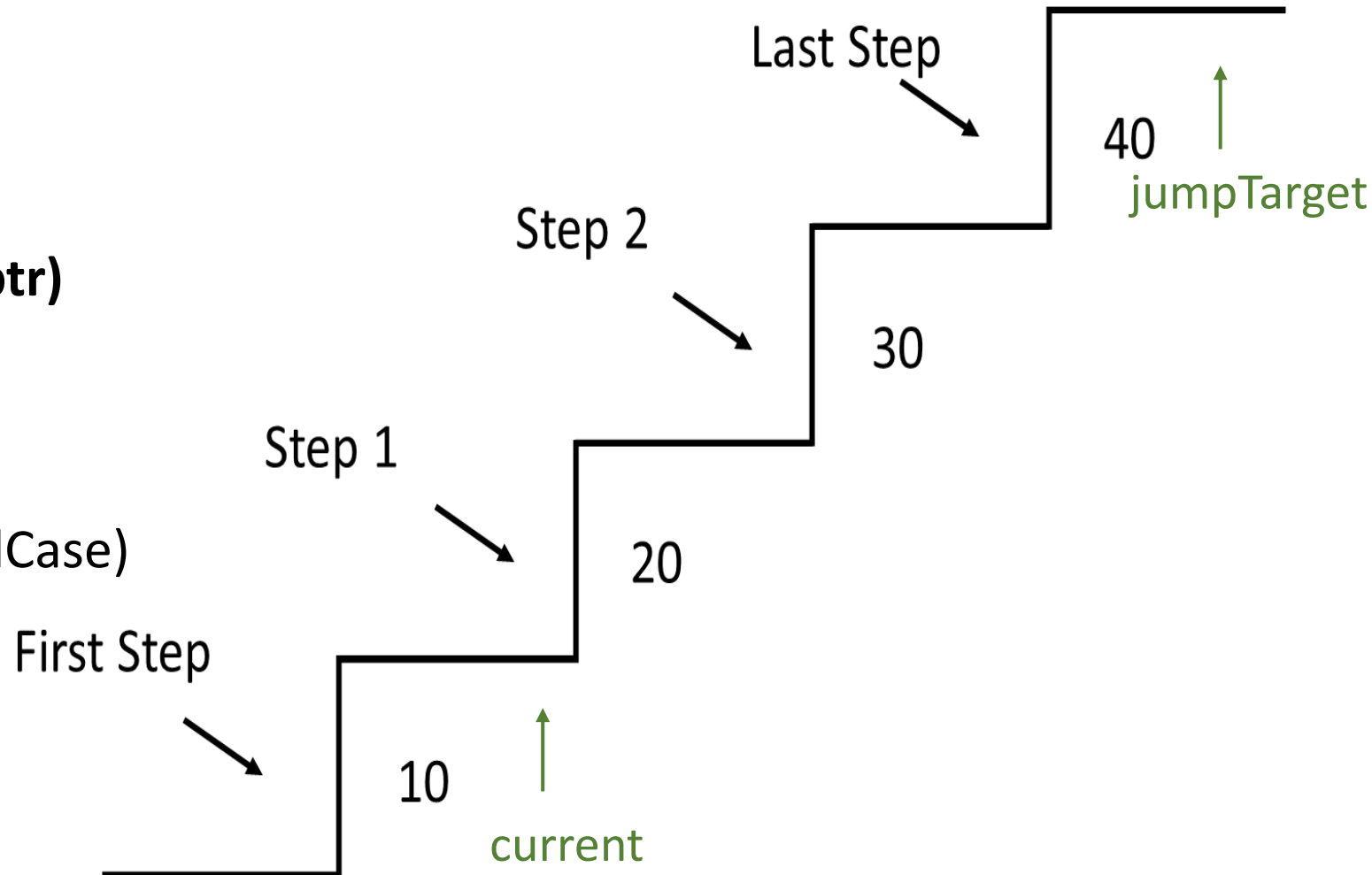
```
    if (step == nullptr)  
        return false;
```

```
    if (step->jumpTarget == nullptr)  
        return false;
```

```
    //[10].jumpTarget -> 40
```

```
    if (userStatu < step->requiredCase)  
        return false;
```

```
    return true;    }
```



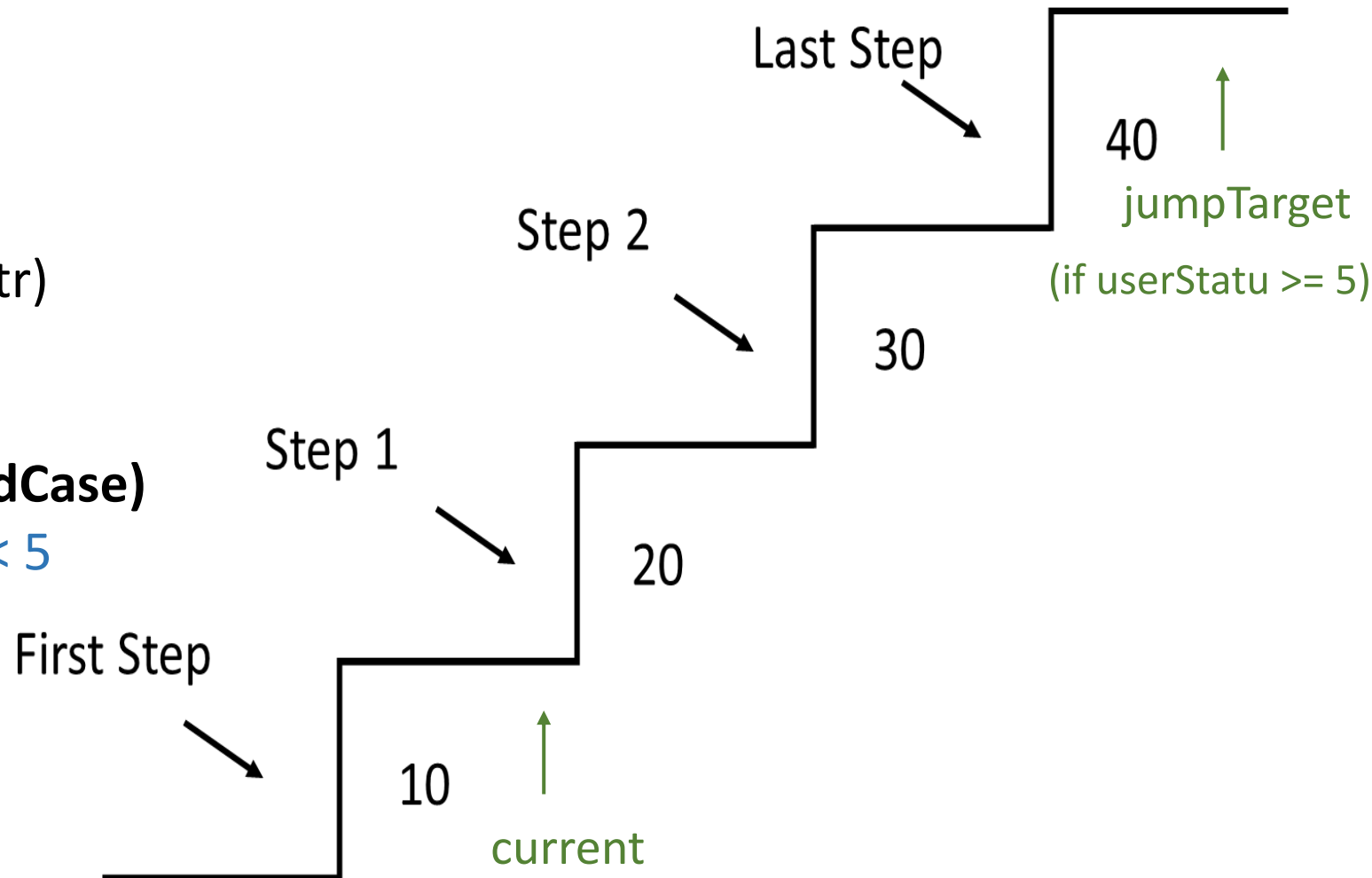
canJump

```
bool canJump(Step* step, int userStatu)
{
    if (step == nullptr)
        return false;

    if (step->jumpTarget == nullptr)
        return false;

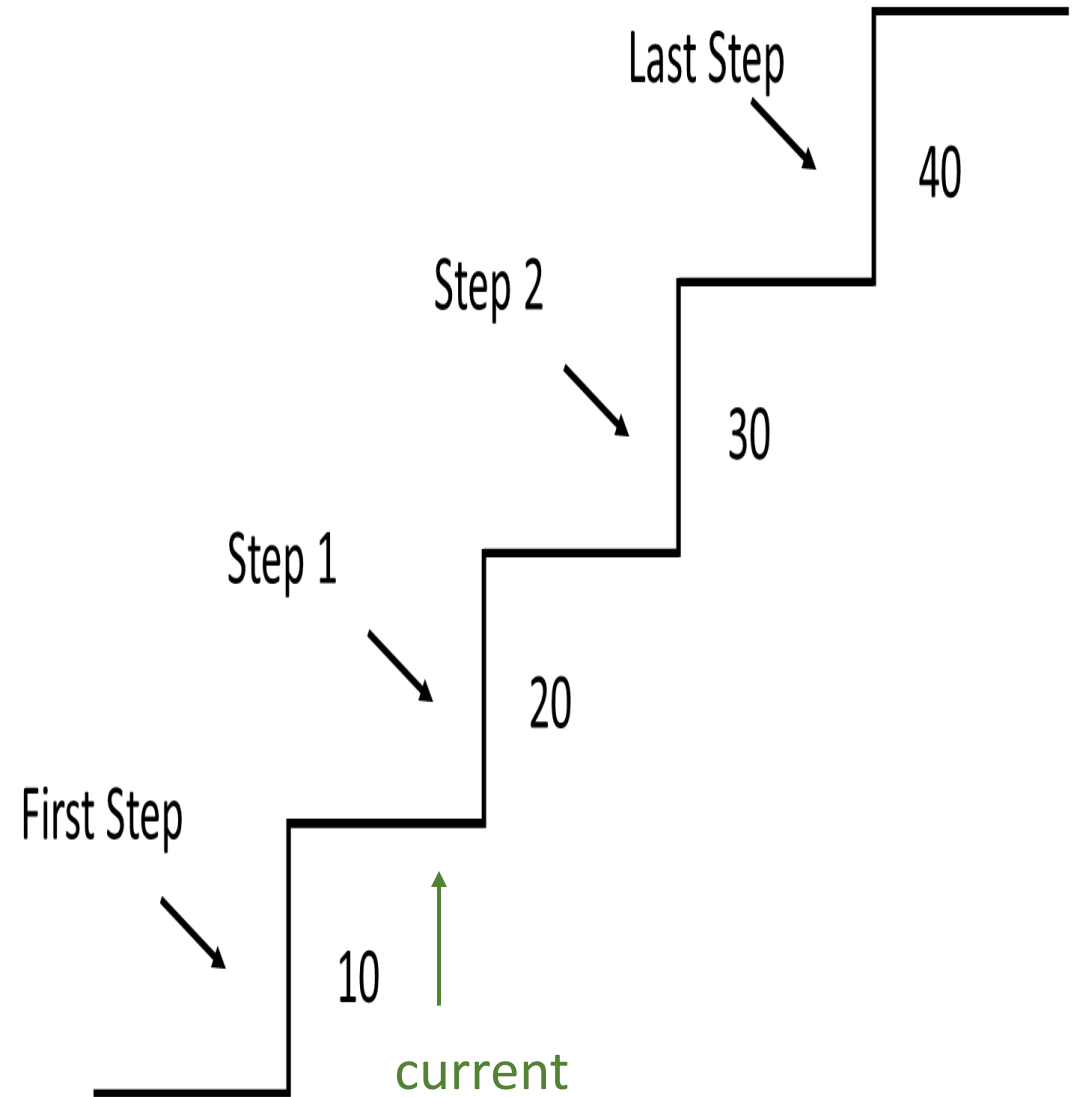
    if (userStatu < step->requiredCase)
        return false; //userStatu < 5

    return true; }
```



Conditional Jump (tryJump)

```
void tryJump(int userStatu)    {  
    if (current == nullptr) { // current = 10  
        cout << "Current is null. Cannot jump.\n";  
        return;  
    }  
  
    if (canJump(current, userStatu))    {  
        cout << "Jump allowed from step " << current->stepNo  
            << " to step " << current->jumpTarget->stepNo << ".\n";  
        current = current->jumpTarget;  
        cout << "Current step after jump: " << current->stepNo <<  
            "\n";  
    }  
    else  
    {  
        cout << "Jump NOT allowed at step " << current->stepNo  
            << ". Continue with normal +1 / -1 steps.\n";    }  
}
```

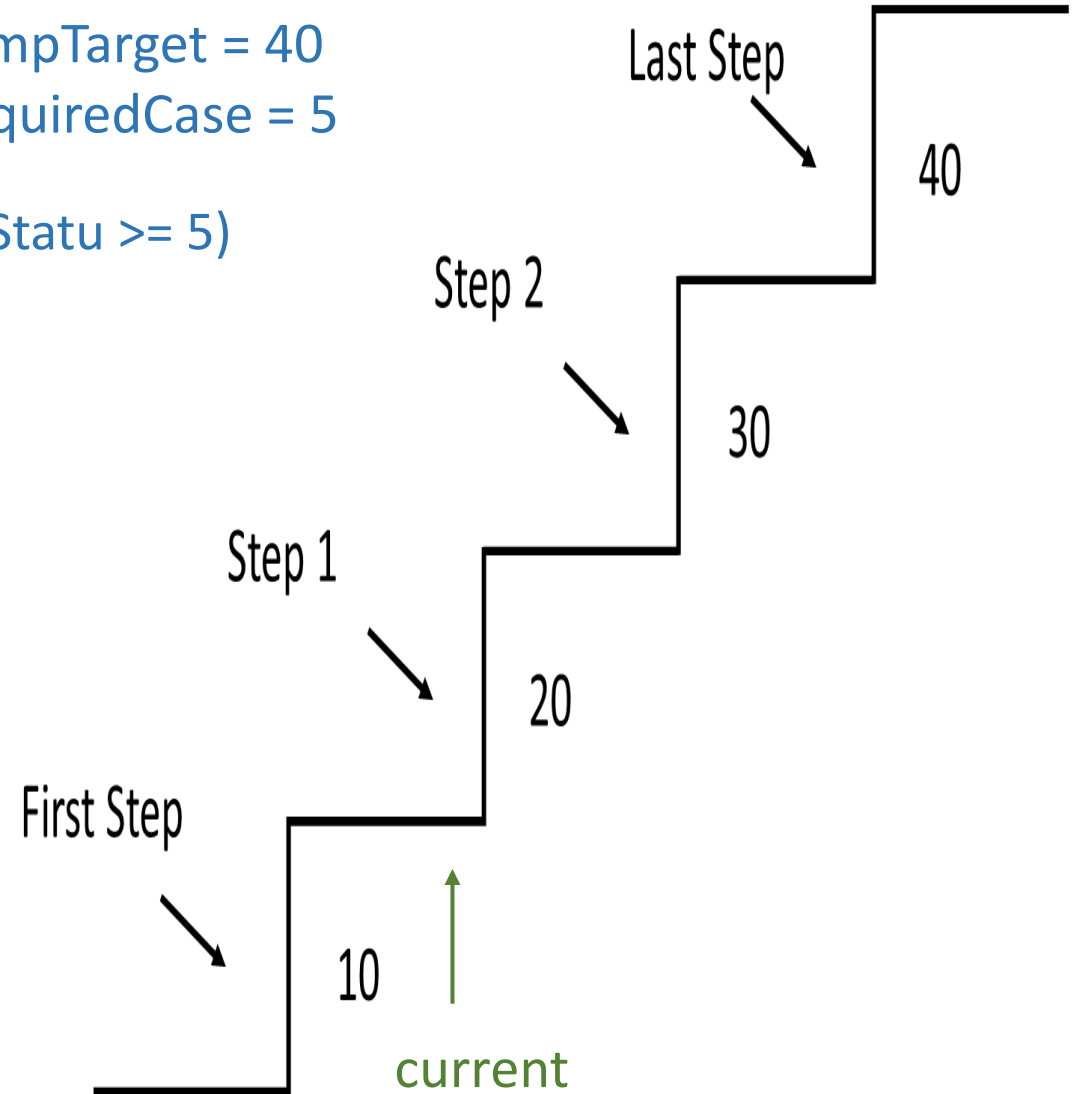


tryJump

```
void tryJump(int userStatu)    {  
    if (current == nullptr)    {  
        cout << "Current is null. Cannot jump.\n";  
        return;  
    }  
  
    if (canJump(current, userStatu))    {  
        cout << "Jump allowed from step " << current->stepNo  
            << " to step " << current->jumpTarget->stepNo << ".\n";  
        current = current->jumpTarget;  
        cout << "Current step after jump: " << current->stepNo <<  
            "\n";  
    }  
    else  
    {  
        cout << "Jump NOT allowed at step " << current->stepNo  
            << ". Continue with normal +1 / -1 steps.\n";    }  
}
```

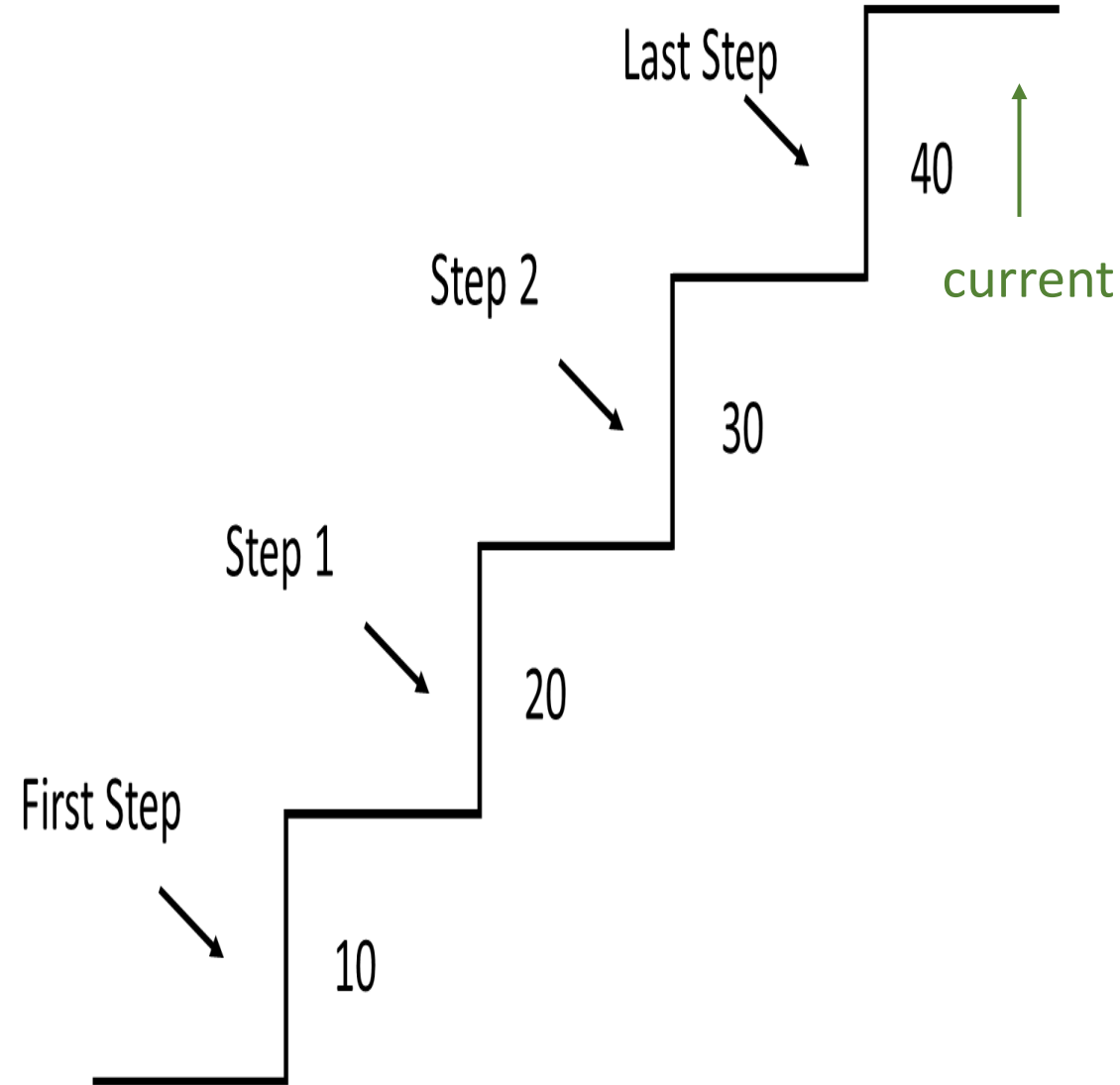
current = 10
[10].jumpTarget = 40
[10].requiredCase = 5

(if userStatu >= 5)



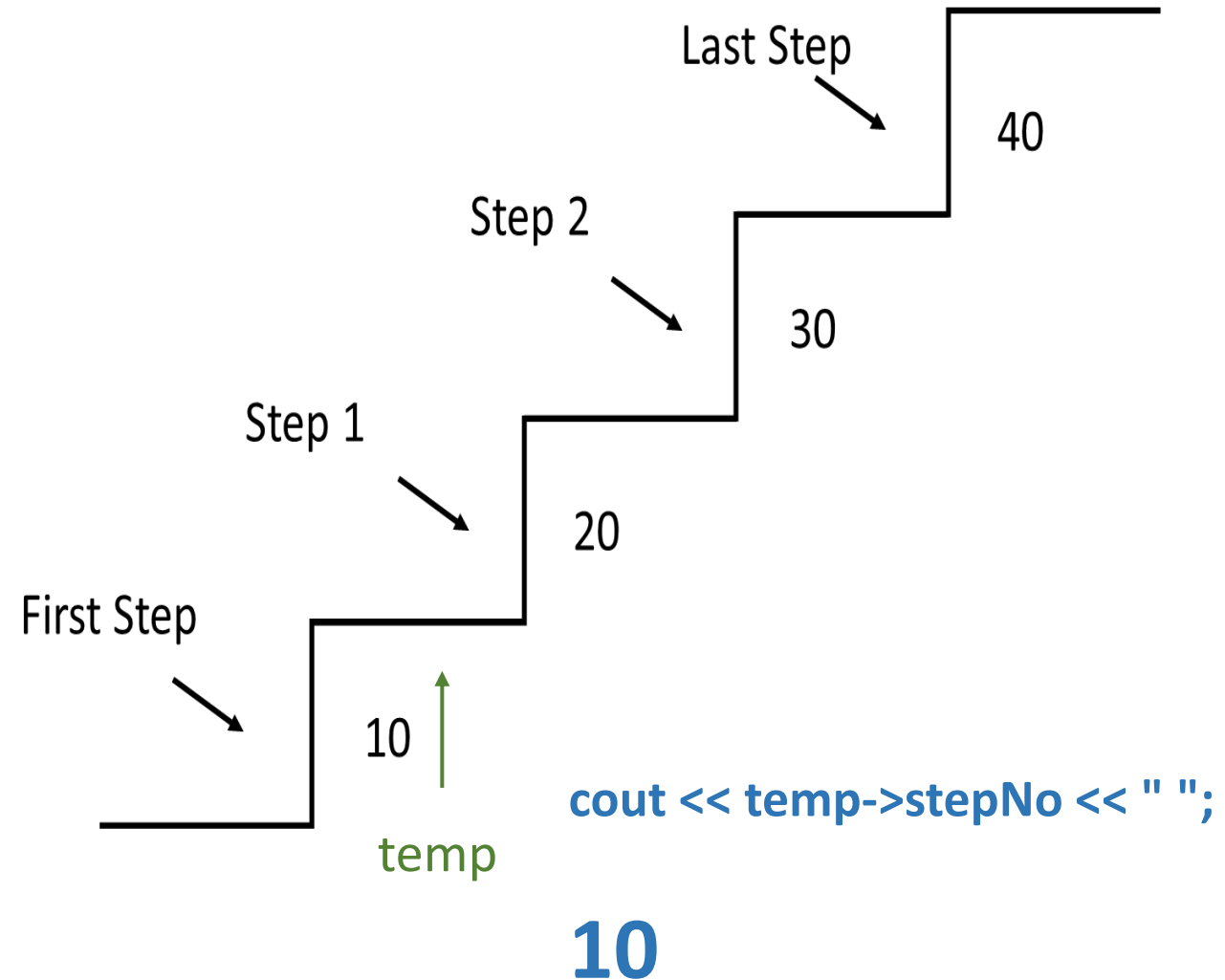
tryJump

```
void tryJump(int userStatu)    {  
    if (current == nullptr)    {  
        cout << "Current is null. Cannot jump.\n";  
        return;  
    }  
  
    if (canJump(current, userStatu))    {  
        cout << "Jump allowed from step " << current->stepNo  
            << " to step " << current->jumpTarget->stepNo << ".\n";  
        current = current->jumpTarget;    // current = 40  
        cout << "Current step after jump: " << current->stepNo <<  
            "\n";  
    }  
    else  
    {  
        cout << "Jump NOT allowed at step " << current->stepNo  
            << ". Continue with normal +1 / -1 steps.\n";    }  
}
```



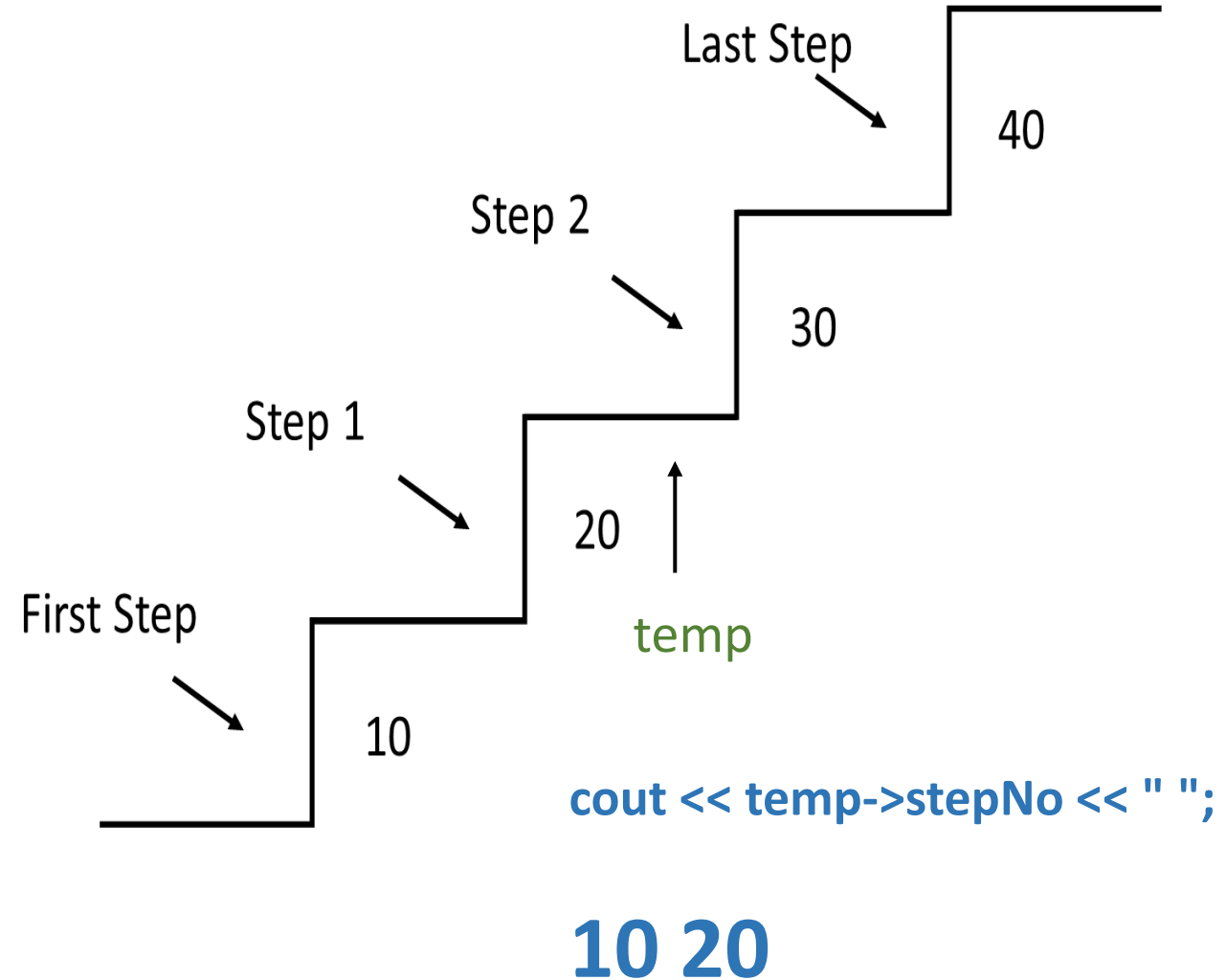
traverseFromBottom

```
void traverseFromBottom() {  
    cout << "Staircase (bottom to top): ";  
  
    for (Step* temp = firstStep; temp !=  
        nullptr; temp = temp->up)  
    {  
        cout << temp->stepNo << " ";  
    }  
  
    cout << "\n";  
}
```



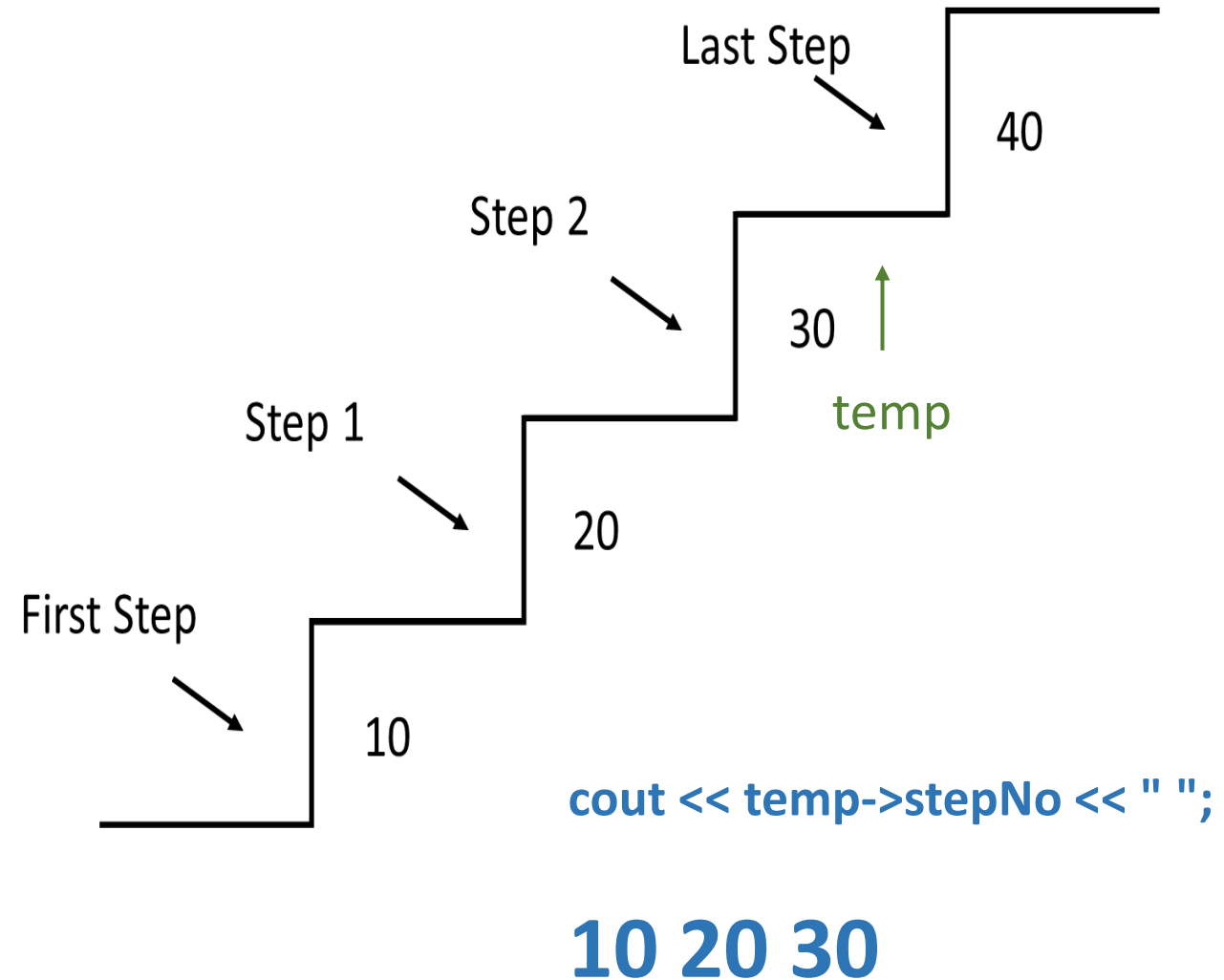
traverseFromBottom

```
void traverseFromBottom() {  
    cout << "Staircase (bottom to top): ";  
  
    for (Step* temp = firstStep; temp !=  
        nullptr; temp = temp->up)  
    {  
        cout << temp->stepNo << " ";  
    }  
  
    cout << "\n";  
}
```



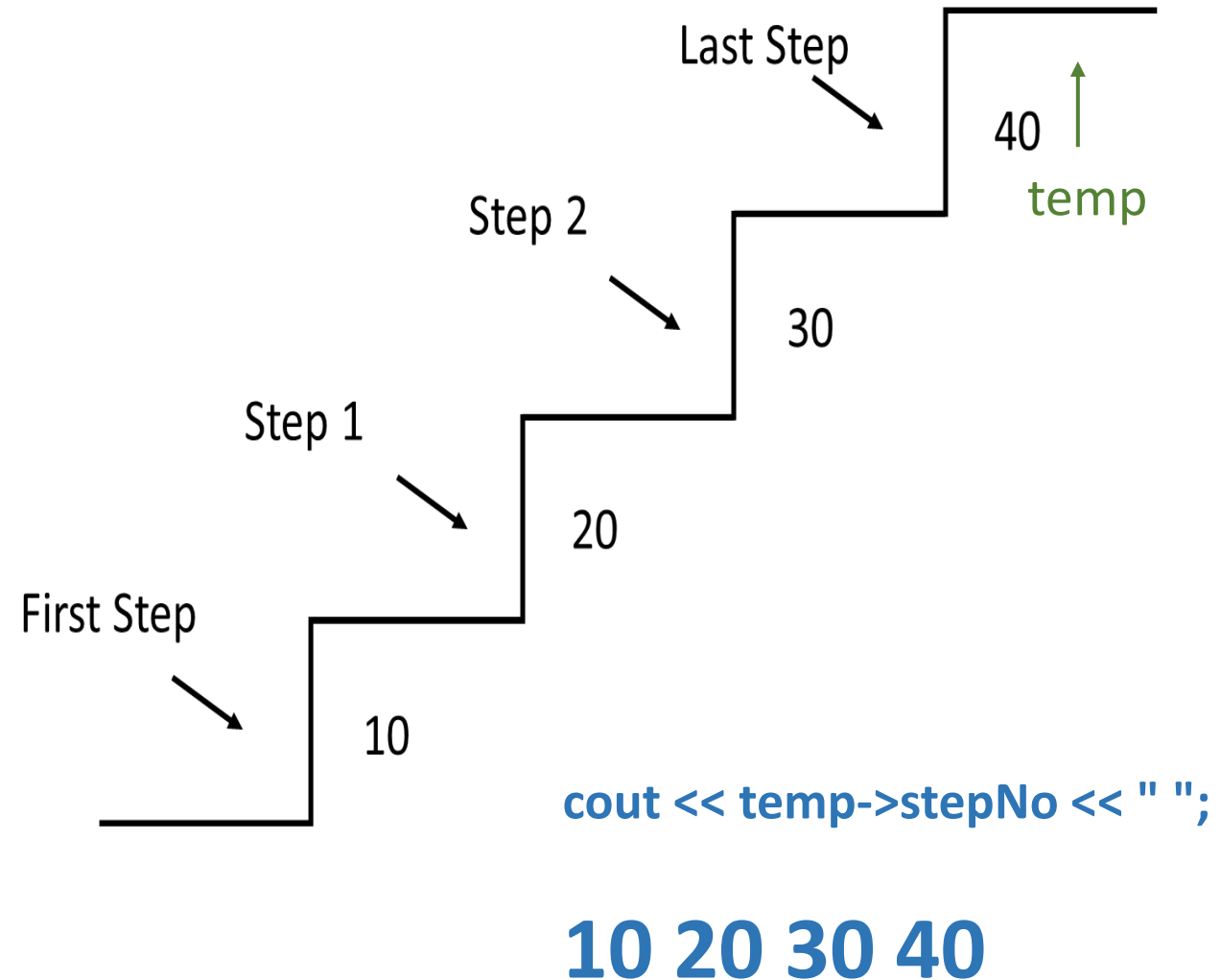
traverseFromBottom

```
void traverseFromBottom() {  
    cout << "Staircase (bottom to top): ";  
  
    for (Step* temp = firstStep; temp !=  
        nullptr; temp = temp->up)  
    {  
        cout << temp->stepNo << " ";  
    }  
  
    cout << "\n";  
}
```



traverseFromBottom

```
void traverseFromBottom() {  
    cout << "Staircase (bottom to top): ";  
  
    for (Step* temp = firstStep; temp !=  
        nullptr; temp = temp->up)  
    {  
        cout << temp->stepNo << " ";  
    }  
  
    cout << "\n";  
}
```

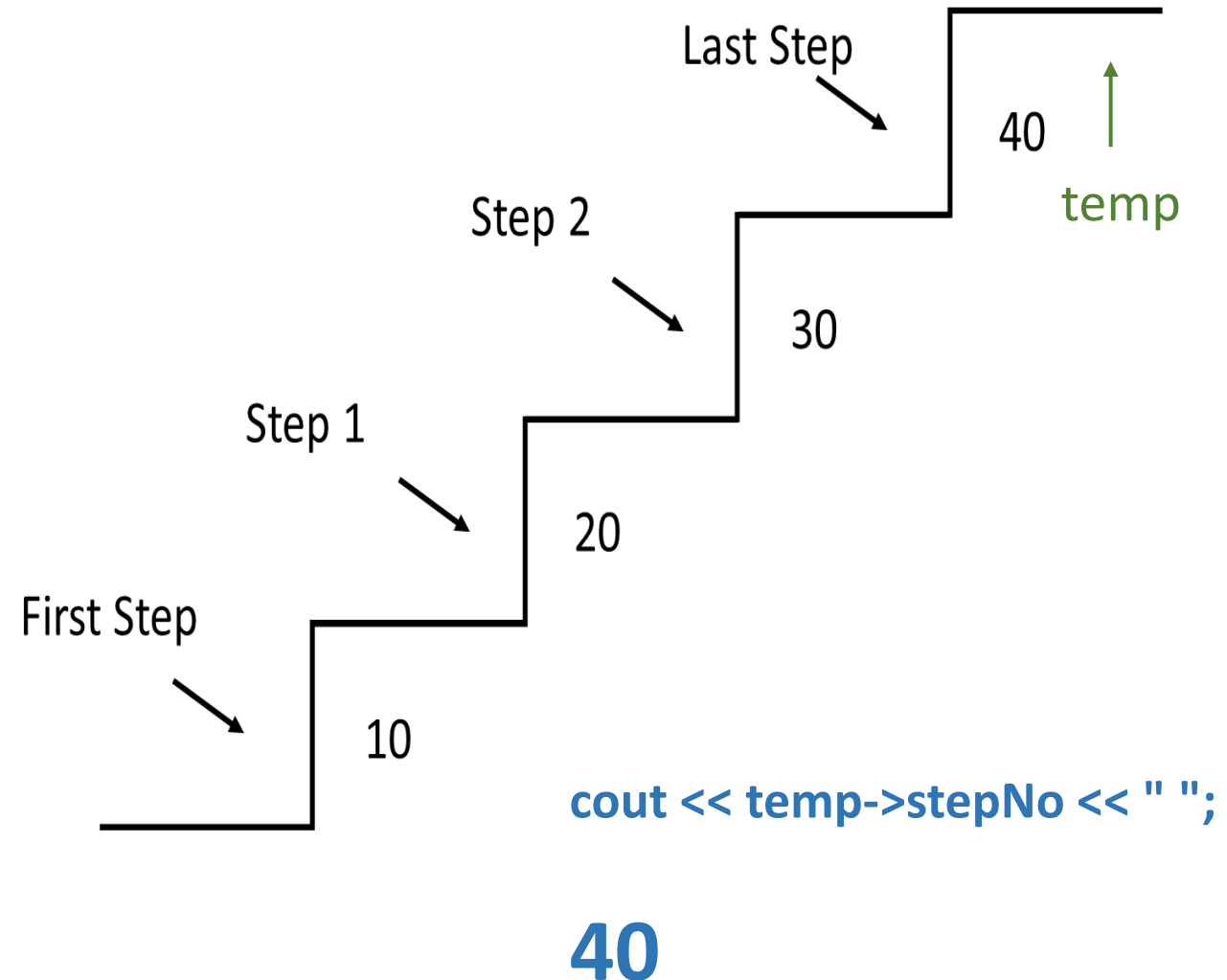


traverseFromTop

```
void traverseFromTop()    // Prints
data from last to first step
{
    cout << "Staircase (top to bottom): ";

    for (Step* temp = lastStep; temp !=
nullptr; temp = temp->down)
    {
        cout << temp->stepNo << " ";
    }

    cout << "\n";
}
```

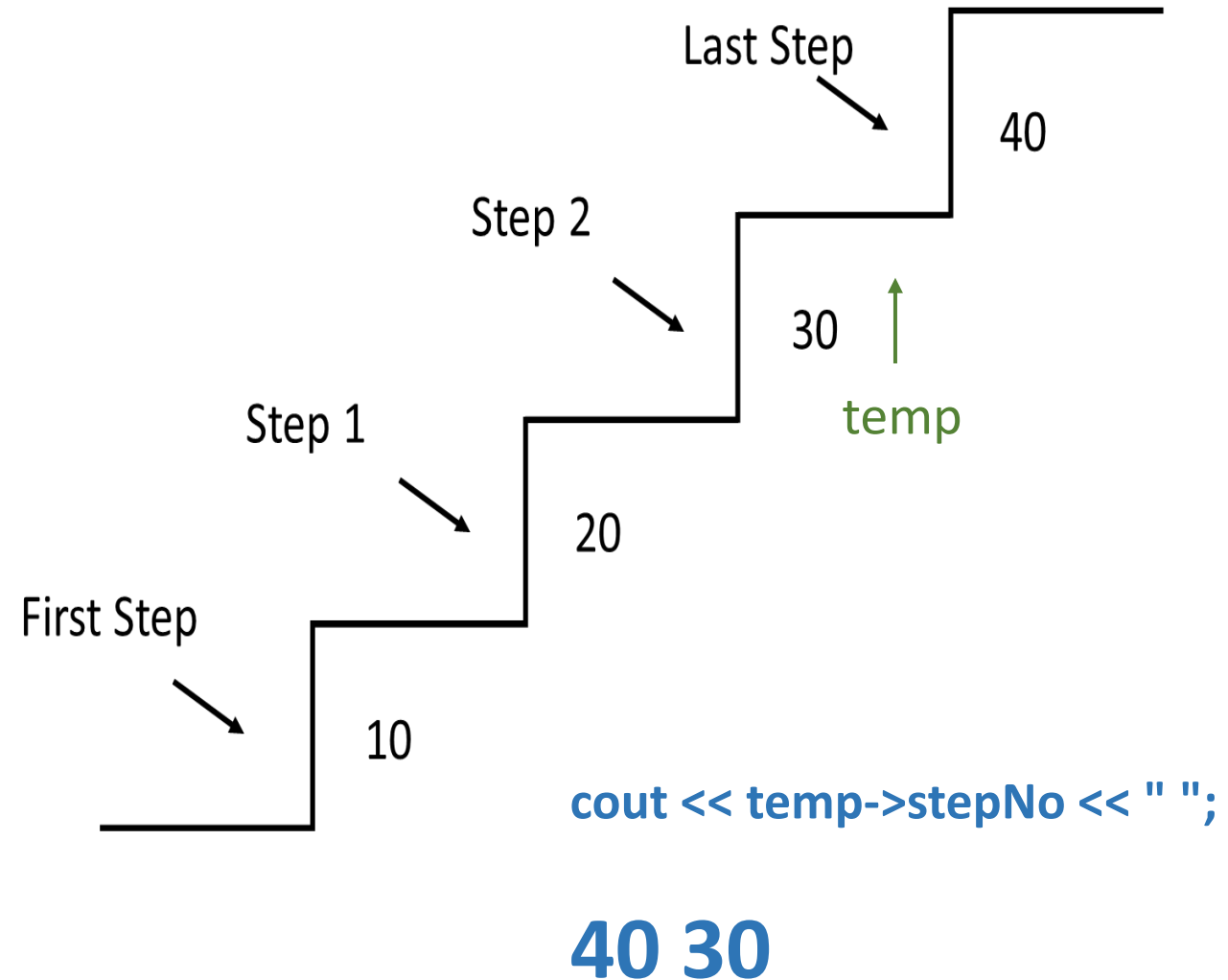


traverseFromTop

```
void traverseFromTop()    // Prints
data from last to first step
{
    cout << "Staircase (top to bottom): ";

    for (Step* temp = lastStep; temp !=
        nullptr; temp = temp->down)
    {
        cout << temp->stepNo << " ";
    }

    cout << "\n";
}
```

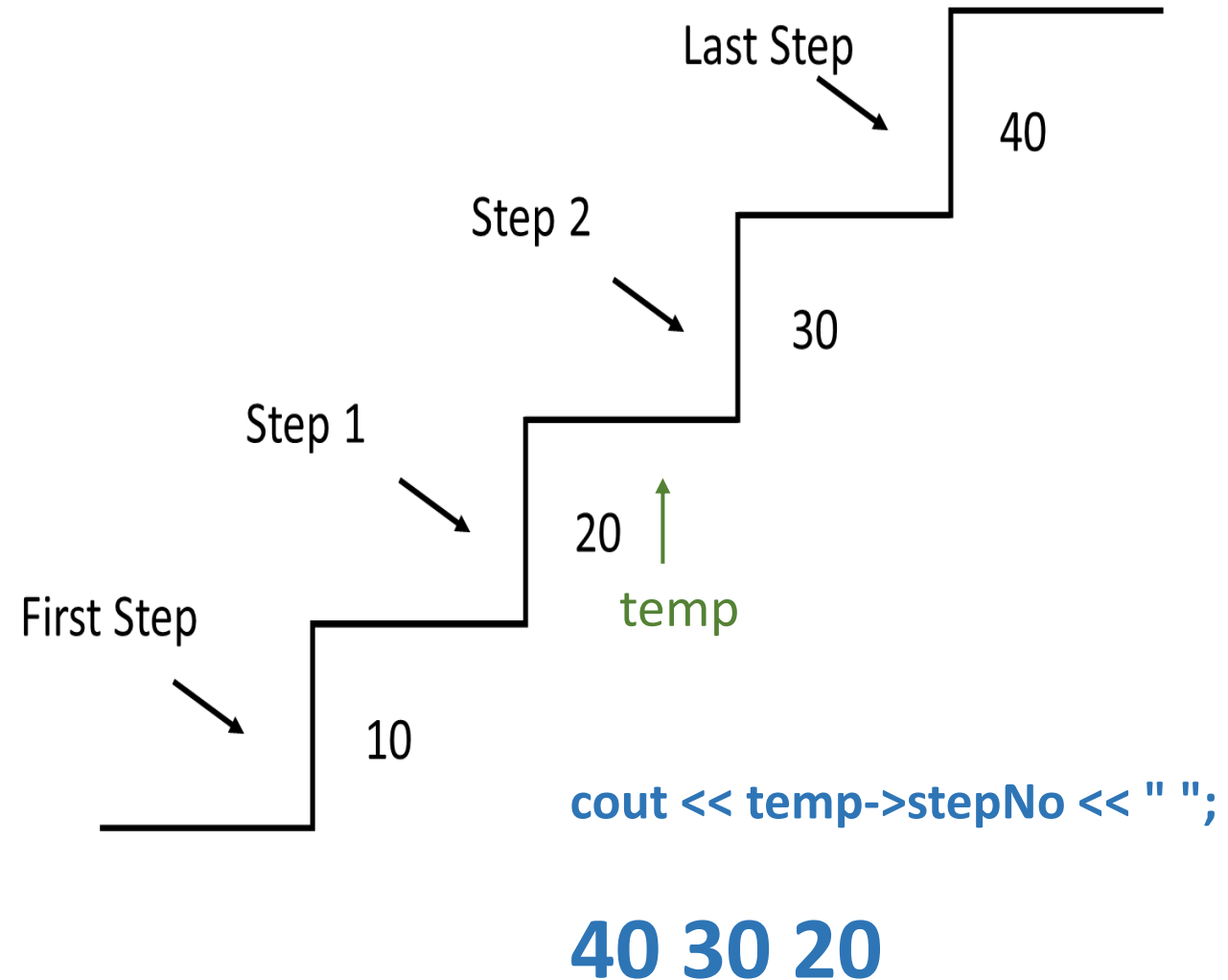


traverseFromTop

```
void traverseFromTop()    // Prints
data from last to first step
{
    cout << "Staircase (top to bottom): ";

    for (Step* temp = lastStep; temp !=
nullptr; temp = temp->down)
    {
        cout << temp->stepNo << " ";
    }

    cout << "\n";
}
```

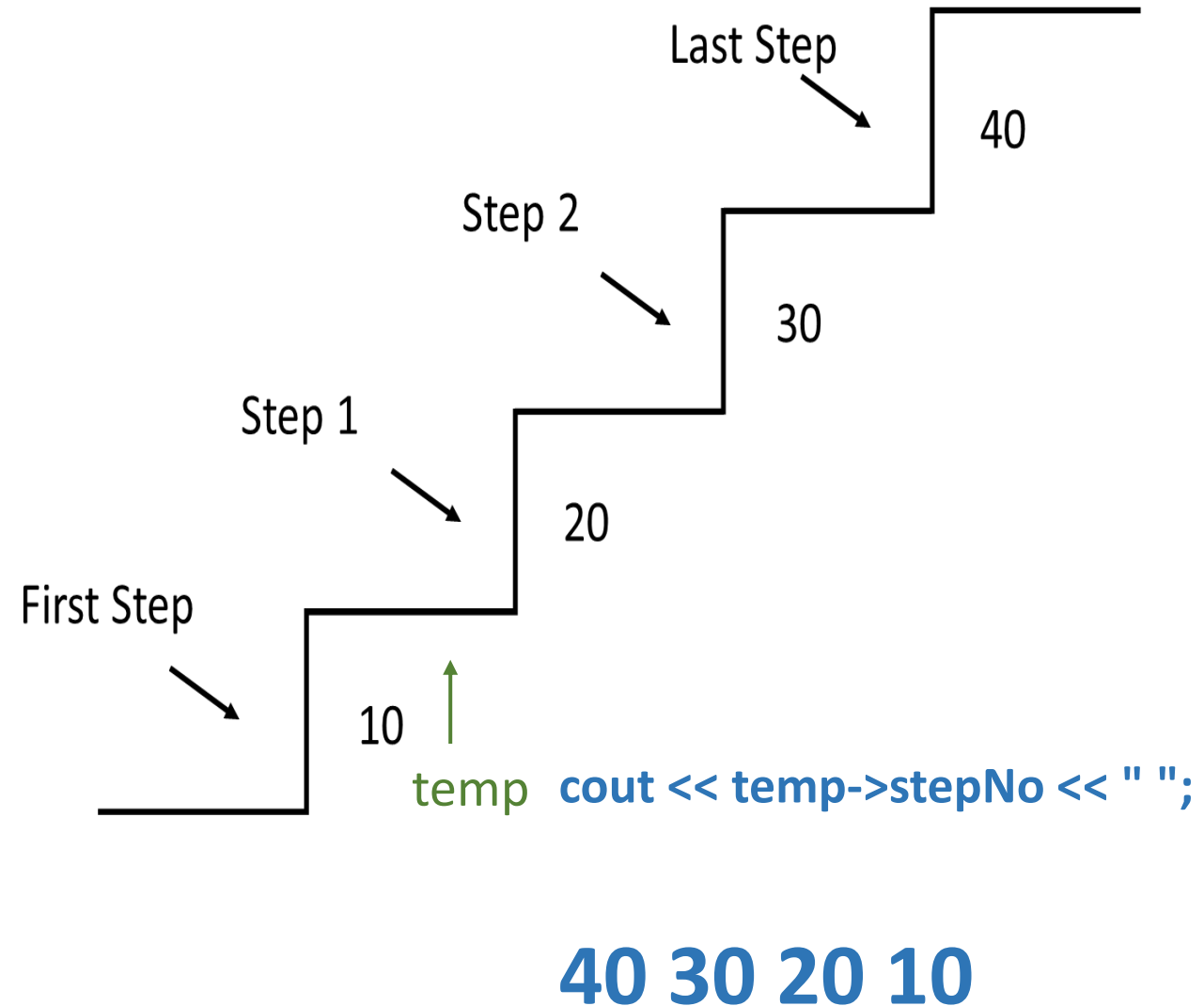


traverseFromTop

```
void traverseFromTop()    // Prints
data from last to first step
{
    cout << "Staircase (top to bottom): ";

    for (Step* temp = lastStep; temp !=
nullptr; temp = temp->down)
    {
        cout << temp->stepNo << " ";
    }

    cout << "\n";
}
```



Time Complexity(Big-O Notation)

Operation	Time Complexity	Explanation
findStep()	$O(n)$	Searches step by value
createStaircase()	$O(n)$	You create n steps and connect them together
insertStep()	$O(n)$	If the position is known, step addition is done in constant time
deleteStep()	$O(n)$	If the node to be deleted is known, the connections are rearranged
moveUp()	$O(1)$	Moving to the next step takes constant time
moveDown()	$O(1)$	Descending to the previous step takes constant time
setJump()	$O(n)$	Defines the condition for jumping
canJump()	$O(1)$	Rule Engine checks condition
tryJump()	$O(1)$	JumpTarget pointer is reached in one go
traverseFromBottom()	$O(n)$	Iterates through all steps; n steps
traverseFromTop()	$O(n)$	Iterates through all steps; n steps

Advantage of CSDS

- Compared to Queue (FIFO): Only allows fixed, sequential processing.
- CSDS: Allows both sequential progression and condition-based multi-step skipping.
- Advantage: CSDS provides flexible progression.
- Compared to Stack (LIFO): Supports only last-in-first-out access.
- CSDS: Supports controlled forward and backward navigation.
- Advantage: CSDS is designed for progression, not just data removal.
- Comparison with Doubly Linked Lists: Only allows incremental progression.
- CSDS: Extends this structure with conditional jump links.
- Advantage: Both normal progression and jump progression are present in a single structure.

Disadvantage of CSDS

- **Compared to Queue (FIFO):** Has a simple and fixed rule (first in, first out).
- **CSDS:** Requires correct definition of jump conditions.
- **Disadvantage:** If conditions are defined incorrectly, the progression logic may behave undesirably.
- **Compared to Stack (LIFO):** Has a very simple control logic.
- **CSDS:** Uses additional rule checks for progression.
- **Disadvantage:** More complex than LIFO.
- **Comparison with Doubly Linked Lists:** Node deletion affects only local pointer connections.
- **CSDS:** Deleting a step may invalidate existing jumpTarget references.
- **Disadvantage:** Jump points targeting a deleted step may need to be rewritten.